第五章 动态规划

Chapter 5 Dynamic programming

计算机学院 潘永生

5.1 动态规划算法总体思想

- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题。
- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。
- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。

Those who cannot remember the past are doomed to repeat it.

-----George Santayana,
The life of Reason,
Book I: Introduction and
Reason in Common
Sense (1905)

最优解包含子问题最优解

一、最优子结构性质

- 最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。

- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。

二、重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**重叠子问题**。

- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。

- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。

求解斐波那契数列的递归算法

$$Fib(n)=\begin{cases} n, & n=0,1 \\ Fib(n-1)+Fib(n-2), & n>1 \end{cases}$$

```
int count=1;                    //累计调用的步骤
int Fib(int n)                  //算法
{  printf("(%d)求解Fib(%d)\n",  count++,  n);
   if (n==1 || n==2)
   {  printf("   计算出Fib(%d)=%d\n",  n,  1);
      return 1;
   }
   else
   {  int x=Fib(n-1);
      int y=Fib(n-2);
      printf("   计算出Fib(%d)=Fib(%d)+Fib(%d)=%d\n",
                                          n,  n-1,  n-2,  x+y);
      return x+y;
   }
}
```

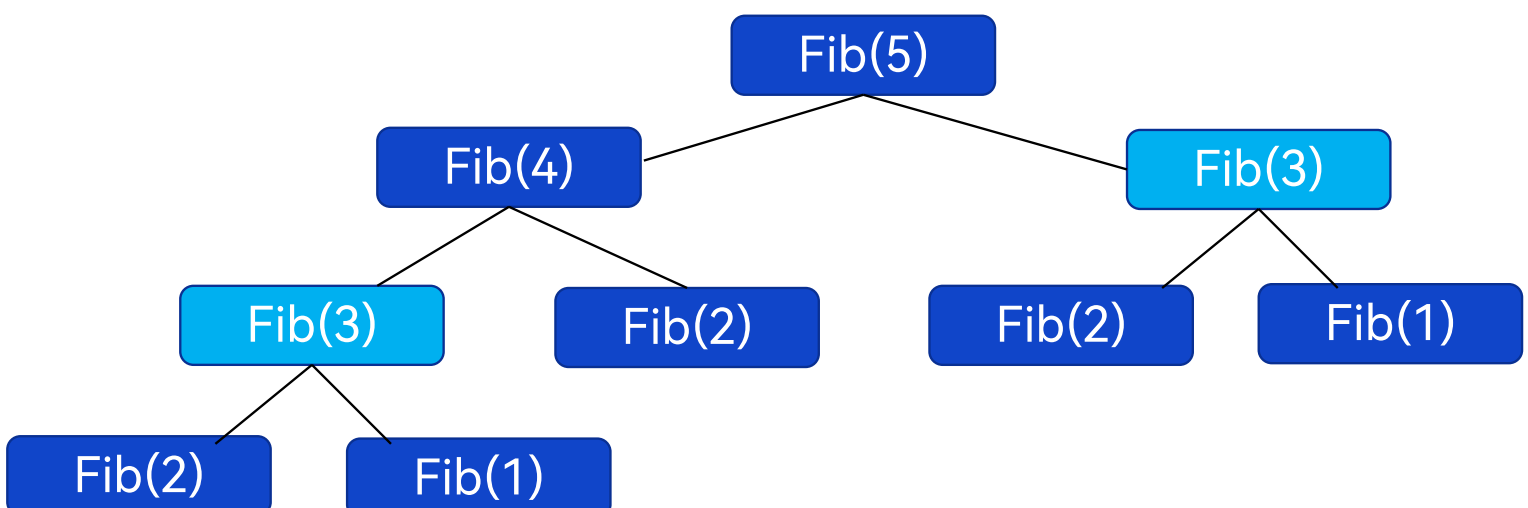
Fib1(5)时的输出结果：

（1）求解Fib(5)
（2）求解Fib(4)
（3）求解Fib(3)
（4）求解Fib(2)
计算出Fib(2)=1
（5）求解Fib(1)
计算出Fib(1)=1
计算出Fib(3)=Fib(2)+Fib(1)=2
（6）求解Fib(2)
计算出Fib(2)=1
计算出Fib(4)=Fib(3)+Fib(2)=3
（7）求解Fib(3)
（8）求解Fib(2)
计算出Fib(2)=1
（9）求解Fib(1)
计算出Fib(1)=1
计算出Fib(3)=Fib(2)+Fib(1)=2
计算出Fib(5)=Fib(4)+Fib(3)=5

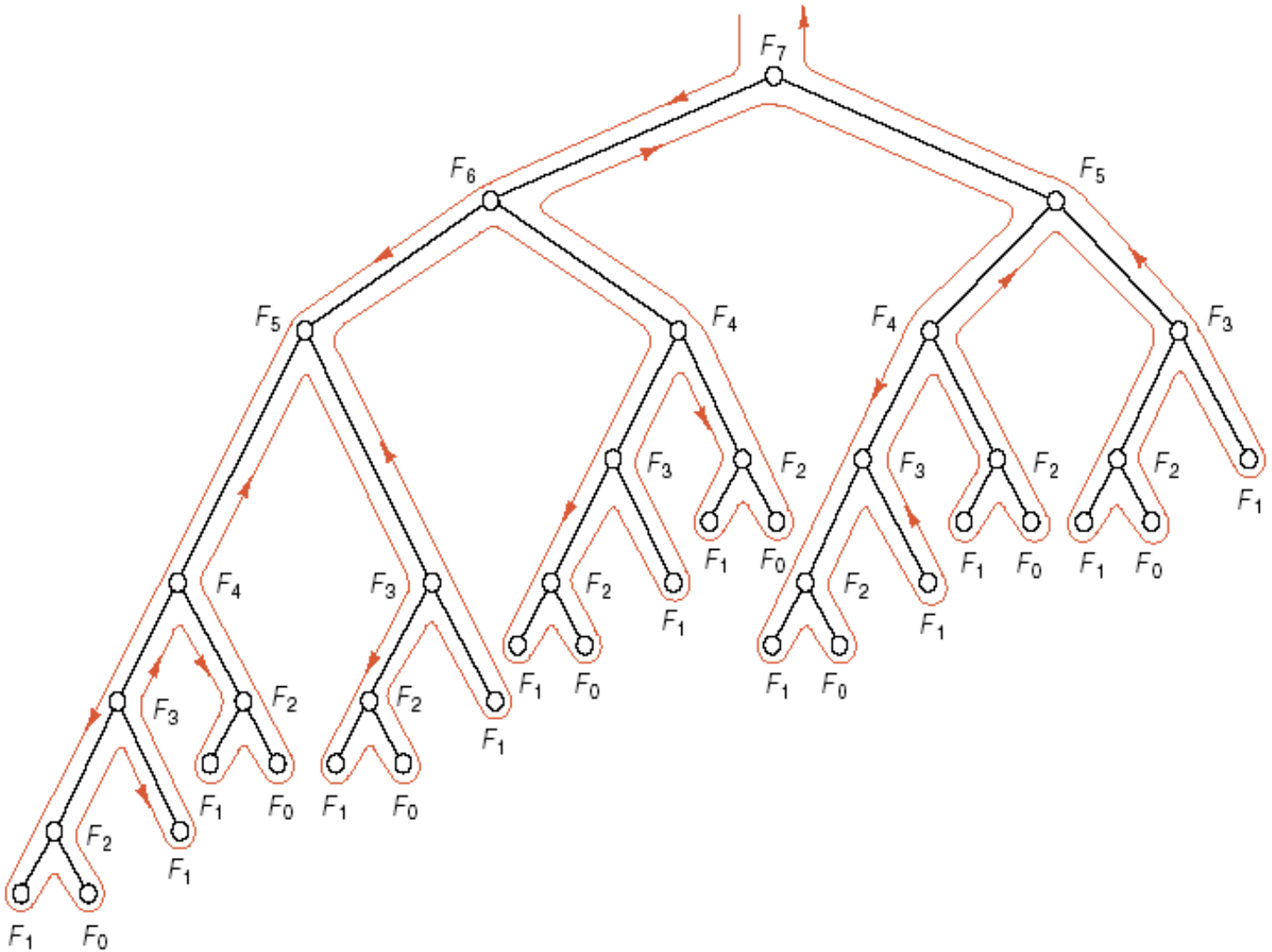
从中看出如下几点：

（1）递归调用Fib(5)采用自顶向下的执行过程，从调用Fib(5)开始到计算出Fib(5)结束。

（2）计算过程中存在大量的重复计算，例如求Fib(5)的过程如图所示，存在两次重复计算Fib(3)值的情况。



5.2 斐波那契查序列


F7
F6
F5
F5
F4
F4
F3
F4
F3
F3
F2
F3
F2
F2
F2
F1
F1
F0
F1
F0
F1
F0
F1
F0
F2
F2
F2
F1
F1
F0
F1
F0
F3
F2
F2
F1
F1
F0
F1
F0
F2
F1
F1
F0

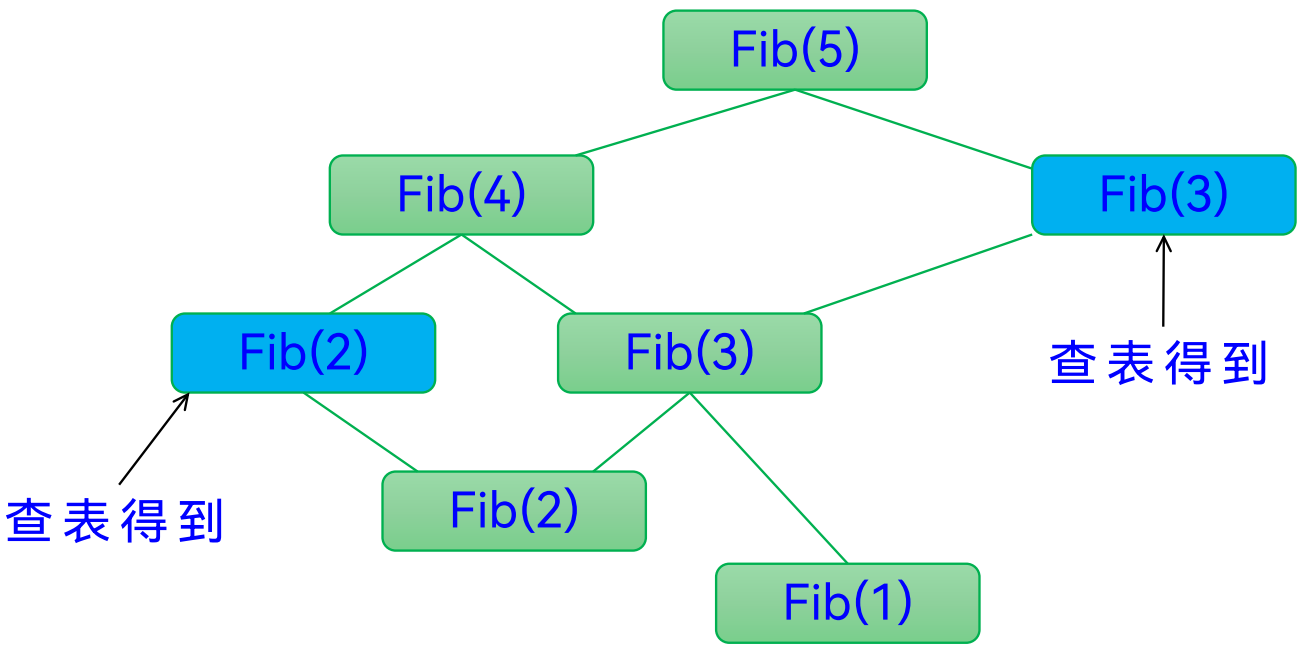
为此避免重复计算，设计一个dp数组，dp[i]存放Fib(i)的值，首先设置dp[1]和dp[2]均为1，再让i从3到n循环以计算dp[3]到dp[n]的值，最后返回dp[n]即Fib1(n)。对应的算法1如下：

```
int dp[MAX];                                        //所有元素初始化为0
int count=1;                                        //累计调用的步骤
int Fib1(int n)                                 //算法1
{  dp[1]=dp[2]=1;
   printf("(%d)计算出Fib(1)=1\n",  count++);
   printf("(%d)计算出Fib(2)=1\n",  count++);
   for (int i=3;i<=n;i++)
   {  dp[i]=dp[i-1]+dp[i-2];
      printf("(%d)计算出Fib(%d)=%d\n",  count++,  i,  dp[i]);
   }
   return dp[n];
}
```

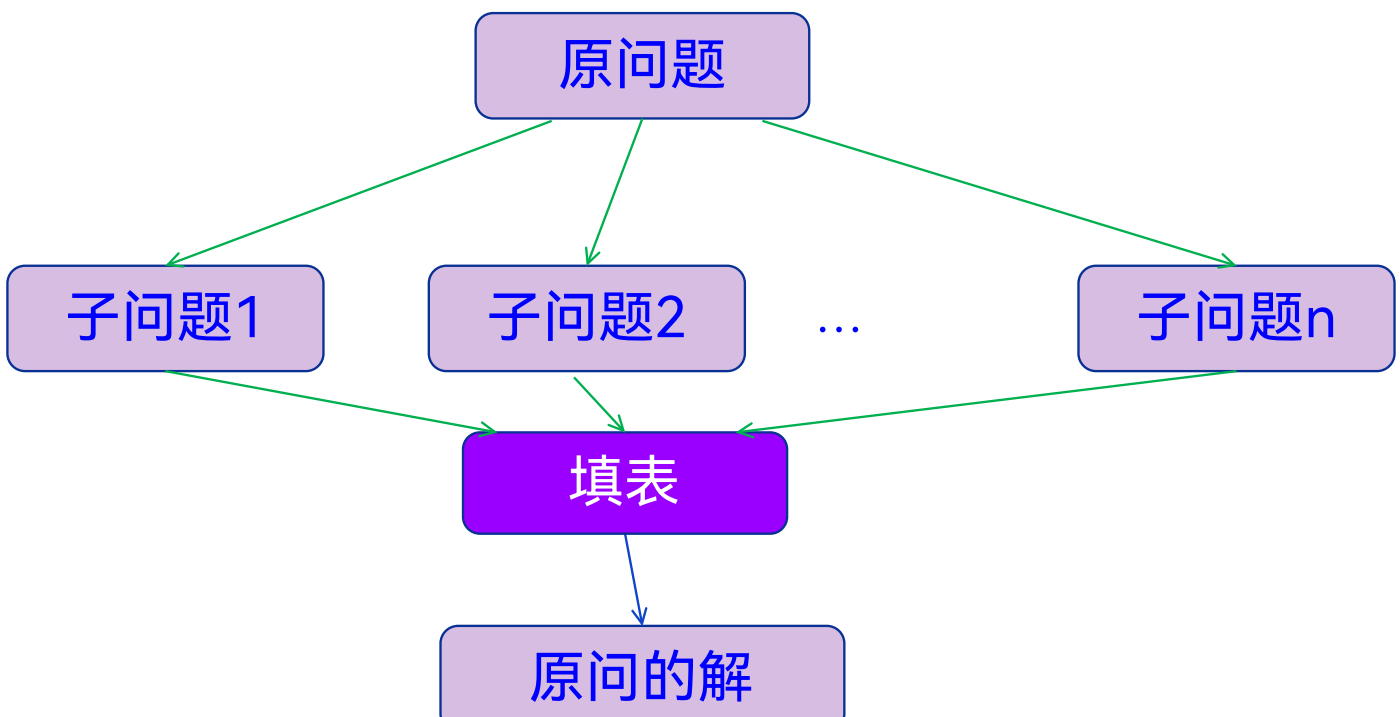
执行Fib1(5)时的输出结果如下:

(1) 计算出Fib1(1)=1
(2) 计算出Fib1(2)=1
(3) 计算出Fib1(3)=2
(4) 计算出Fib1(4)=3
(5) 计算出Fib1(5)=5

其执行过程改变为自底向上，即先求出子问题解，将计算结果存放在一张表中，而且相同的子问题只计算一次，在后面需要时只有简单查表，以避免大量的重复计算。



上述求斐波那契数列的算法1属于动态规划法，其中数组dp（表）称为动态规划数组。动态规划法也称为记录结果再利用的方法，其基本求解过程如下图所示。



【问题描述】字符序列的子序列是指从给定字符序列中随意地（不一定连续）去掉若干个字符（可能一个也不去掉）后所形成的字符序列。

令给定的字符序列$X=(x_0, x_1, \ldots, x_{m-1})$，序列$Y=(y_0, y_1, \ldots, y_{k-1})$是$X$的子序列，存在$X$的一个严格递增下标序列$(i_0, i_1, \ldots, i_{k-1})$，使得对所有的$j=0, 1, \ldots, k-1$，有 $x_{i_j} = y_j$

例如，$X=(a, b, c, b, d, a, b)$，$Y=(b, c, d, b)$是X的一个子序列。

给定两个序列A和B，称序列Z是A和B的公共子序列，是指Z同是A和B的子序列。该问题是求两序列A和B的最长公共子序列（LCS）。

【问题求解】若设$A=(a_0, a_1, \ldots, a_{m-1})$，$B=(b_0, b_1, \ldots, b_{n-1})$，设$Z=(z_0, z_1, \ldots, z_{k-1})$为它们的最长公共子序列。不难证明有以下性质:

- 如果$a_{m-1}=b_{n-1}$，则$z_{k-1}=a_{m-1}=b_{n-1}$，且$(z_0, z_1, \ldots, z_{k-2})$是$(a_0, a_1, \ldots, a_{m-2})$和$(b_0, b_1, \ldots, b_{n-2})$的一个最长公共子序列。
- 如果$a_{m-1}\neq b_{n-1}$且$z_{k-1}\neq a_{m-1}$，则$(z_0, z_1, \ldots, z_{k-1})$是$(a_0, a_1, \ldots, a_{m-2})$和$(b_0, b_1, \ldots, b_{n-1})$的一个最长公共子序列。
- 如果$a_{m-1}\neq b_{n-1}$且$z_{k-1}\neq b_{n-1}$，则$(z_0, z_1, \ldots, z_{k-1})$是$(a_0, a_1, \ldots, a_{m-1})$和$(b_0, b_1, \ldots, b_{n-2})$的一个最长公共子序列。

定义二维动态规划数组dp，其中dp[*i*][*j*]为子序列（a_0，a_1，…，a_{i-1}）和（b_0，b_1，…，b_{j-1}）的最长公共子序列的长度。

每考虑一个字符*a*[i]或*b*[j]都为动态规划的一个阶段（共经历约$m\times n$个阶段）。

情况1：*a*[*i*-1]=*b*[*j*-1]

(a_0，a_1，…，a_{i-1})

‖

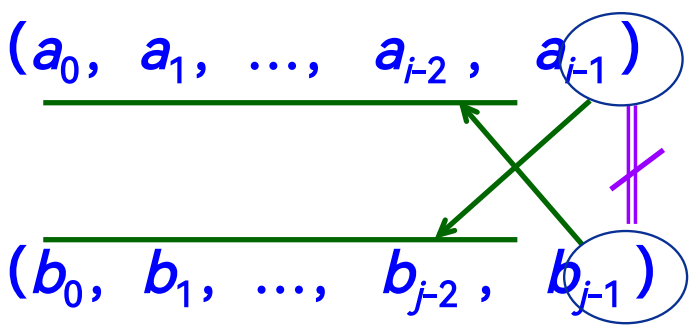
(b_0，b_1，…，b_{j-1})

⇒ dp[*i*][*j*]=dp[*i*-1][*j*-1]+1

定义二维动态规划数组dp，其中dp[*i*][*j*]为子序列（a_0，a_1，…，a_{i-1}）和（b_0，b_1，…，b_{j-1}）的最长公共子序列的长度。

每考虑一个字符*a*[i]或*b*[j]都为动态规划的一个阶段（共经历约*m*×*n*个阶段）。

情况2：*a*[*i*-1]≠*b*[*j*-1]



dp[*i*][*j*]=MAX(dp[*i*][*j*-1]，dp[*i*-1][*j*])

那么如何由dp求出一个LCS呢?

dp[*i*][*j*]=0	*i*=0或*j*=0—边界条件
dp[*i*][*j*]=dp[*i*-1][*j*-1]+1	*a*[*i*-1]=*b*[*j*-1]
dp[*i*][*j*]=MAX(dp[*i*][*j*-1], dp[*i*-1][*j*])	*a*[*i*-1]≠*b*[*j*-1]

❶当dp[*i*][*j*] ≠ dp[*i*][*j*-1]（左边）并且dp[*i*][*j*] ≠ dp[*i*-1][*j*]（上方）值时:

a[*i*-1]=*b*[*j*-1] ⇒ 它是LCS的一个字符

将*a*[*i*-1]添加到LCS中。

❷ 其他情况时: *a*[*i*-1]或者*b*[*j*-1]是LCS的一个字符!

总 结:

用vector<char>字符向量subs存放一个LCS，*k*=dp[*m*][*n*]（LCS的字符个数），从*k*到1循环求出subs中的*k*个字符：

2

（1）如果dp[*i*][*j*]==dp[*i*-1][*j*]（上方），说明*a*[*i*-1]或*b*[*j*-1]不是LCS中的字符 □ *i*--;

（2）如果dp[*i*][*j*]==dp[*i*][*j*-1]（左边），说明*a*[*i*-1]或*b*[*j*-1]不是LCS中的字符 □ *j*--;

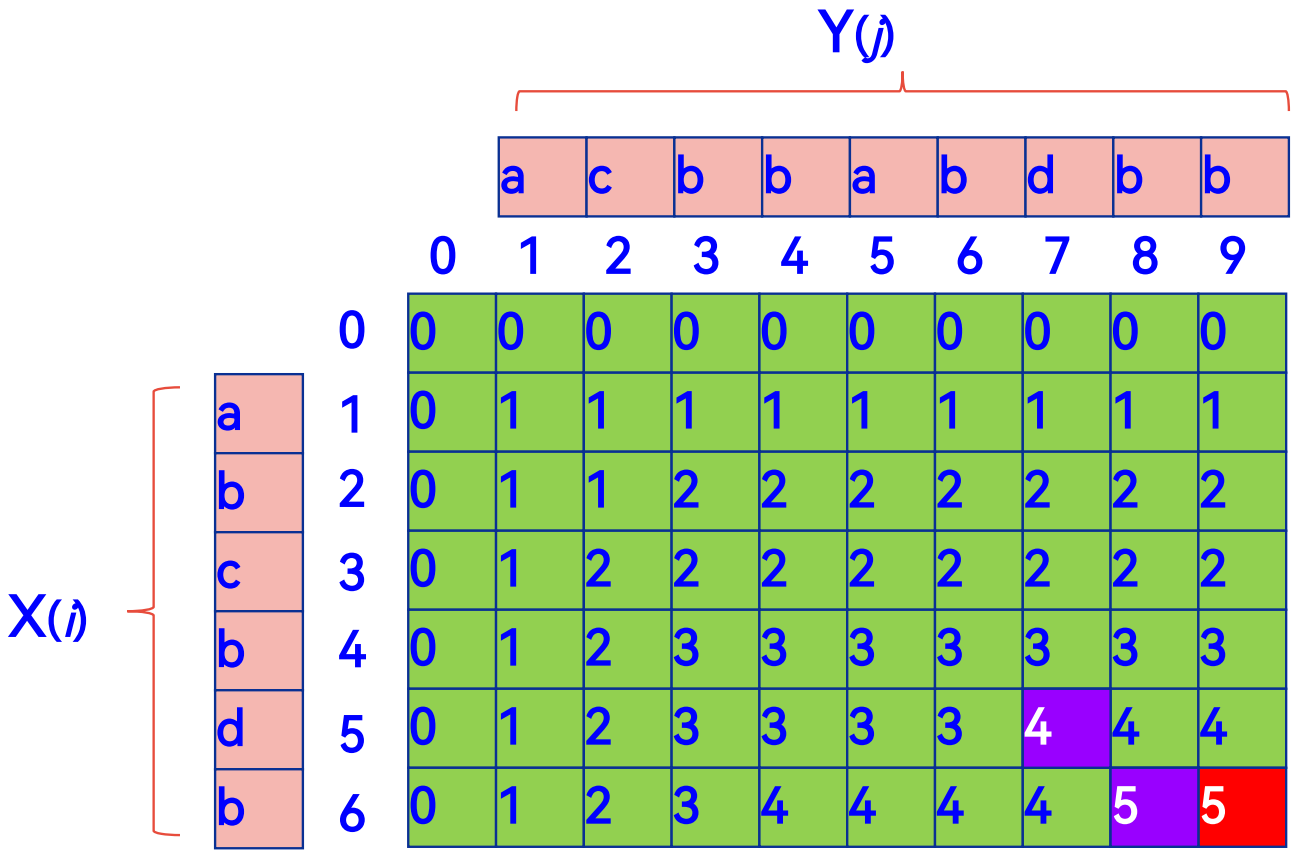
（3）其他情况，说明*a*[*i*-1]或者*b*[*j*-1]是LCS的一个字符!

1

□ *i*--; *j*--; *k*--;表示求的字符减少1个

例如，X=（a，b，c，b，d，b），m=6，Y=（a，c，b，b，a，b，d，b，b），n=9。

（1）求出dp



（2）dp[6][9]=5开始

LCS: d b

i=6，j=9

↓ 与左边相等，j--

i=6，j=8

↓ 与左、上方不等，i--，j--

i=5，j=7

↓ 与左、上方不等，i--，j--

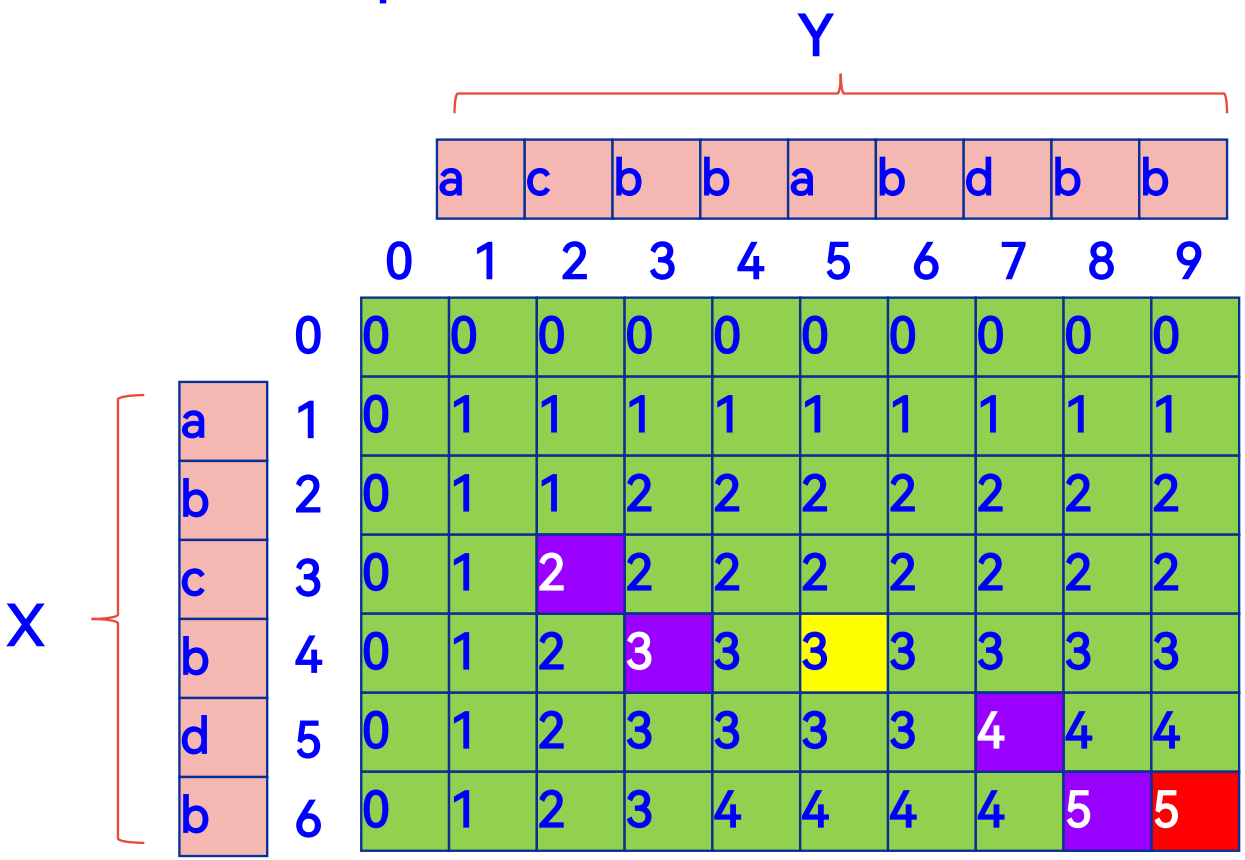
i=4，j=6

↓ 与左边相等，j--

i=4，j=5

那么如何由dp求出LCS呢？例如，X=（a，b，c，b，d，b），m=6，Y=（a，c，b，b，a，b，d，b，b），n=9。

（1）求出dp



（2）dp[6][9]=5开始

LCS: c b d b

i=4，j=5

↓ 与左边相等，j--

i=4，j=4

↓ 与左边相等，j--

i=4，j=3

↓ 与左、上方不等，i--，j--

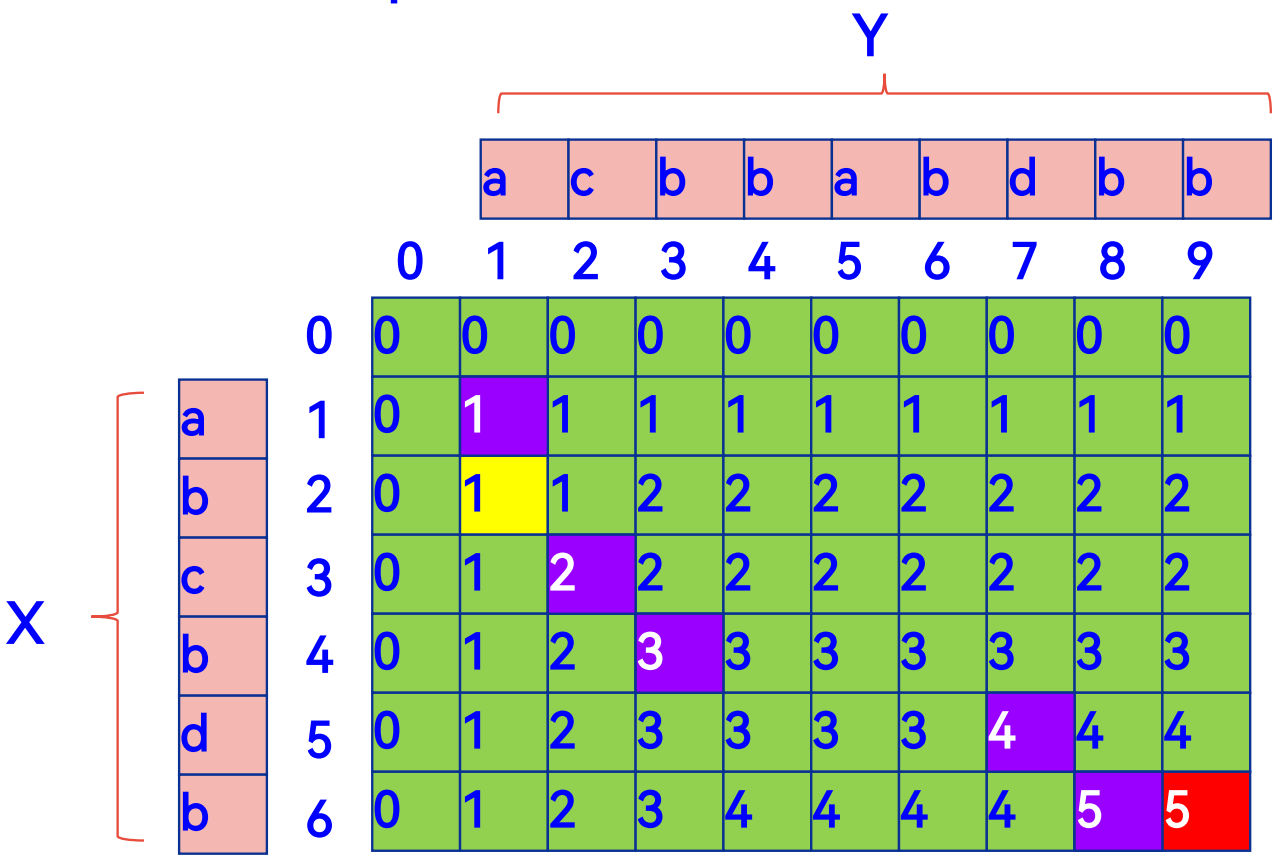
i=3，j=2

↓ 与左、上方不等，i--，j--

i=2，j=1

那么如何由dp求出LCS呢？例如，X=（a，b，c，b，d，b），m=6，Y=（a，c，b，b，a，b，d，b，b），n=9。

（1）求出dp



（2）dp[6][9]=5开始

LCS: a c b d b

i=2，j=1

↓ 与上方相等，i--

i=1，j=1

↓ 与左、上方不等，i--，j--

i=0，j=0

结果LCS="acbdb"

```
#define MAX 51                          //序列中最多的字符个数
//问题表示
int m，n;
string a，b;                            //求解结果表示
int dp[MAX][MAX];                       //动态规划数组
vector<char> subs;                      //存放LCS
```

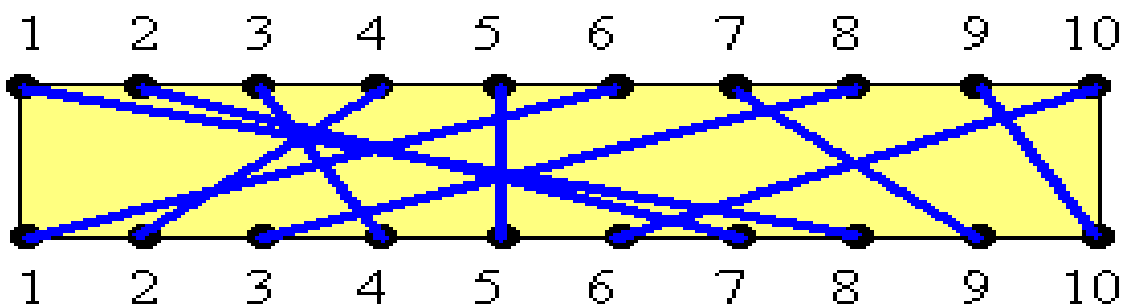
```
void LCSlength()                        //求dp
{ int i，j;
  for (i=0;i<=m;i++)                    //将dp[i][0]置为0，边界条件
    dp[i][0]=0;
  for (j=0;j<=n;j++)                    //将dp[0][j]置为0，边界条件
    dp[0][j]=0;
  for (i=1;i<=m;i++)
    for (j=1;j<=n;j++)                  //两重for循环处理a、b的所有字符
    { if (a[i-1]==b[j-1])               //情况(1)
        dp[i][j]=dp[i-1][j-1]+1;
      else                              //情况(2)
        dp[i][j]=max(dp[i][j-1]，dp[i-1][j]);
    }
}
```

```
void Buildsubs()                    //由dp构造从subs
{  int k=dp[m][n];                  //k为a和b的最长公共子序列长度
   int i=m;
   int j=n;
   while (k>0)                                  //在subs中放入最长公共子序列(反向)
    if (dp[i][j]==dp[i-1][j])
              i--;
    else if (dp[i][j]==dp[i][j-1])
              j--;
    else                                //与上方、左边元素值均不相等
    {         subs.push_back(a[i-1]);  //subs中添加a[i-1]
              i--; j--; k--;
    }
}
```

【算法分析】 LCSlength算法中使用了两重循环，所以对于长度分别为m和n的序列，求其最长公共子序列的时间复杂度为O(m×n)。空间复杂度为O(m×n)。

在一块电路板的上、下2端分别有n个接线柱。根据电路设计，要求用导线(i,π(i))将上端接线柱与下端接线柱相连，如图所示。其中π(i)是{1,2,…,n}的一个排列。导线(i,π(i))称为该电路板上的第i条连线。对于任何1≤i<j≤n，第i条连线和第j条连线相交的充分且必要的条件是π(i)>π(j)。

电路布线问题要确定将哪些连线安排在第一层上，使得该层上有尽可能多的连线。换句话说，该问题要求确定导线集Nets={(i,π(i)),1≤i≤n}的最大不相交子集。



记 N(i,j)的最大不相交子集为MNS(i,j)。Size(i,j)=|MNS(i,j)|。

(1)当i=1时，$N(i,j)=\{t\mid(t,\pi(t))\in Nets,t\le i,\pi(t)\le j\}$

(2)当i>1时，$MNS(1,j)=N(1,j)=\begin{cases}\varnothing & j<\pi(1)\\ \{(1,\pi(1))\} & j\ge\pi(1)\end{cases}$

2.1 j<π(i)。此时，$(i,\pi(i))\notin MNS(i,j)$故在这种情况下，N(i,j)=N(i-1,j)，从而Size(i,j)=Size(i-1,j)。

2.2 j≥π(i)，(i,π(i))∈MNS(i,j) 。则对任意(t,π(t)) ∈MNS(i,j)有t<i且π(t)<π(i)。在这种情况下MNS(i,j)-{(i,π(i))}是N(i-1,π(i)-1)的最大不相交子集。

2.3 若$(i,\pi(i))\notin N(i,j)$ 则对任意(t,π(t)) ∈MNS(i,j)有t<i。从而 $MNS(i,j)\subseteq N(i-1,j)$ 因此，Size(i,j)≤Size(i-1,j)

另一方面 $MNS(i-1,j)\subseteq N(i,j)$ 故又有Size(i,j)≥Size(i-1,j)

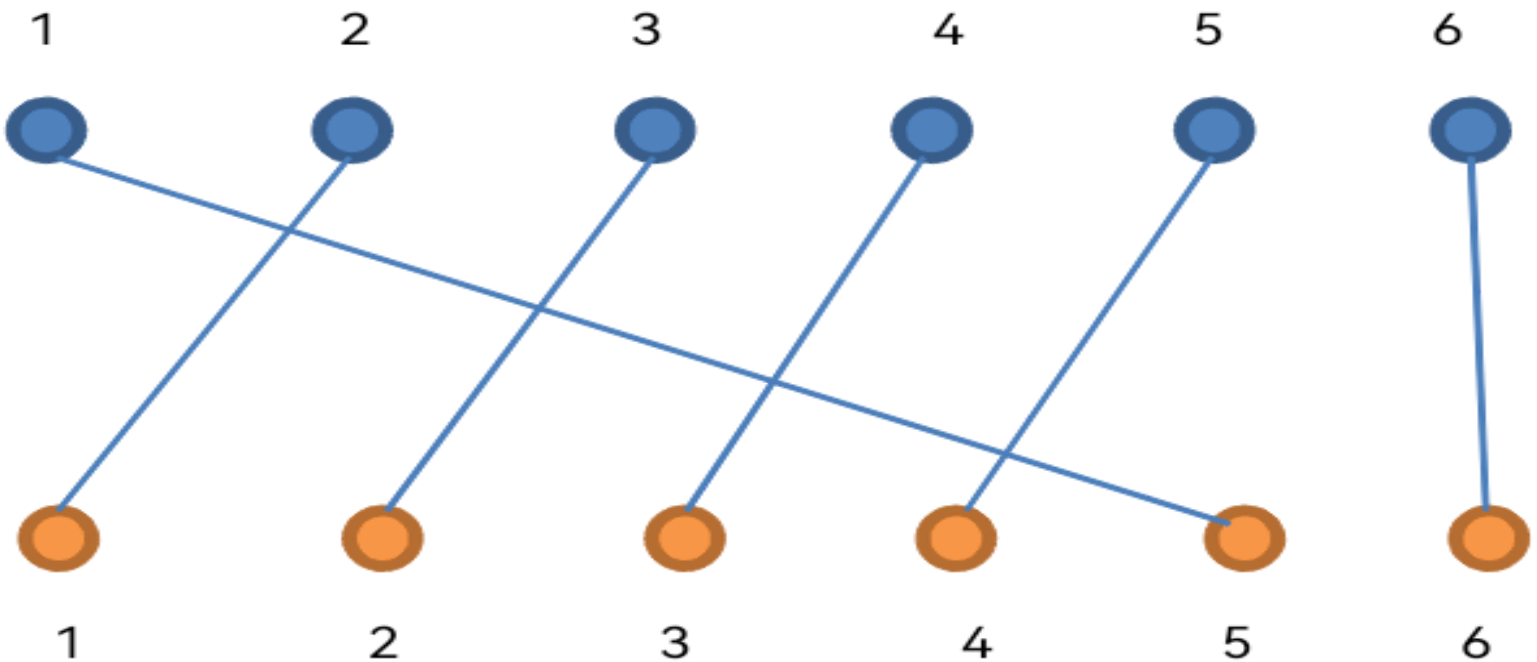
从而Size(i,j)=Size(i-1,j)。

(1)当i=1时

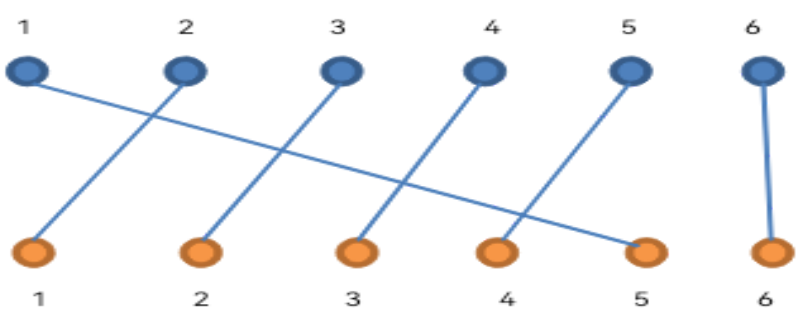
$$Size(1, j) = \begin{cases} 0 & j < \pi(1) \\ 1 & j \geq \pi(1) \end{cases}$$

(2)当i>1时

$$Size(i, j) = \begin{cases} Size(i-1, j) & j < \pi(i) \\ \max\{ Size(i-1, j), Size(i-1, \pi(i)-1) + 1\} & j \geq \pi(i) \end{cases}$$


1
2
3
4
5
6
1
2
3
4
5
6

i \ j	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	1
2	0	1	1	1	1	1	1
3	0	1	2	2	2	2	2
4	0	1	2	3	3	3	3
5	0	1	2	3	4	4	4
6	0	1	2	3	4	4	5

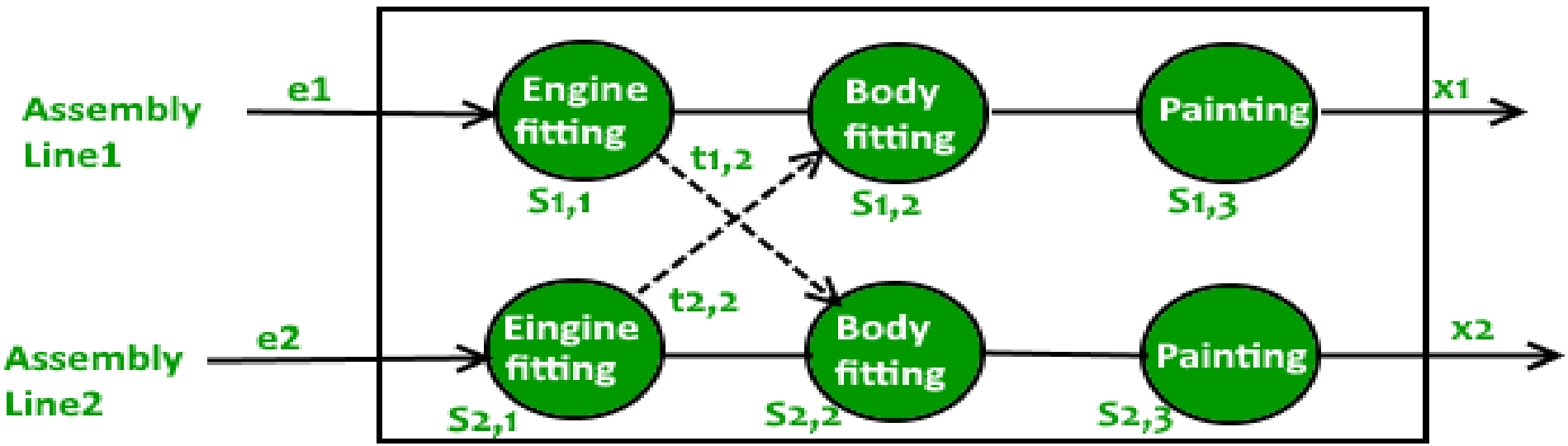


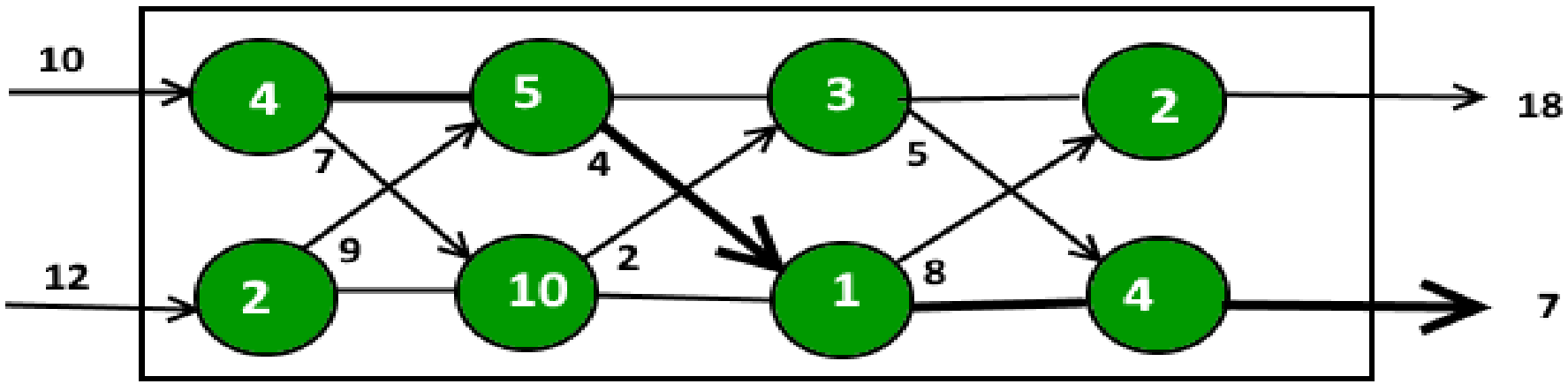
```
void memset(int[]C,int [][]size)
{
    int n=c.length-1;
    for(int j=0;j<C[1];j++)
    {
            size[1][j]=0;
    }

    for(int j=C[1]; j<=n; j++)
    {
        size[1][j]=1;
    }
}
```

```
for(int i=2; i<n; i++)
{
    for(int j=0; j<C[i]; j++)
    {
        size[i][j]=size[i-1][j];//当j<c[i]的情形
    }
    for(int j=C[i]; j<=n; j++)
    {
        //当j>=c[i]时，考虑(i，c[i])是否属于MNS(i,j)的两种情况
        size[i][j]=max(size[i-1][j],size[i-1][C[i]-1]+1);
    }
}
size[n][n]=max(size[n-1][n],size[n-1][C[n]-1]+1);
```

```
void Traceback(int []c,int[][]size,int []net,)
{
   int n=c.length-1;
   j=n;
   m=0;
   for(int i=n;i>1;i--)
   {
      if(size[i][j]!=size[i-1][j])//此时，  (i,c[i])是最大不相交子集的一条边
      {
         Net[m++]=i; //记录连线在上排的节点号
         j=C[i]-1;//更新扩展连线柱区间
      }
   }
   if(j>=C[1])//处理i=1的情形
   {
      Net[m++]=1;
   }
}
```

- A car factory has two assembly lines, each with n stations. A station is denoted by $S_{i,j}$ where i is either 1 or 2 and indicates the assembly line the station is on, and j indicates the number of the station. The time taken per station is denoted by $a_{i,j}$. Each station is dedicated to some sort of work like engine fitting, body fitting, painting, and so on. So, a car chassis must pass through each of the n stations in order before exiting the factory. The parallel stations of the two assembly lines perform the same task. After it passes through station $S_{i,j}$, it will continue to station $S_{i,j+1}$ unless it decides to transfer to the other line. Continuing on the same line incurs no extra cost, but transferring from line i at station $j - 1$ to station j on the other line takes time $t_{i,j}$. Each assembly line takes an entry time e_i and exit time x_i which may be different for the two lines. Give an algorithm for computing the minimum time it will take to build a car chassis.


Assembly Line1
e1
Engine fitting
Body fitting
Painting
x1
t1,2
S1,1
S1,2
S1,3
Assembly Line2
e2
Eingine fitting
Body fitting
Painting
x2
t2,2
S2,1
S2,2
S2,3


10
12
4
5
3
2
2
10
1
4
18
7
7
4
5
9
2
8

Breaking the problem into smaller sub-problems:

T1(j) indicates the minimum time taken by the car chassis to leave station j on assembly line 1.

T2(j) indicates the minimum time taken by the car chassis to leave station j on assembly line 2.

Base cases:

$T1(1) = e_1 + a_{1,1}$

$T2(1) = e_2 + a_{2,1}$

Recursive Relations:

The car chassis at station $S_{1,j}$ can come either from station $S_{1,j-1}$ or station $S_{2,j-1}$.

Case #1: Its previous station is $S_{1,j-1}$

The minimum time to leave station $S_{1,j}$ is given by:

$T1(j) = T1(j-1) + a_{1,j}$

Case #2: Its previous station is $S_{2,j-1}$

The minimum time to leave station S1, j is given by:

$T1(j) = T2(j-1) + t_{2,j} + a_{1,j}$

The minimum time T1(j) is given by the minimum of the two obtained in cases #1 and #2.

$T1(j) = \min((T1(j-1) + a_{1,j}), (T2(j-1) + t_{2,j} + a_{1,j}))$

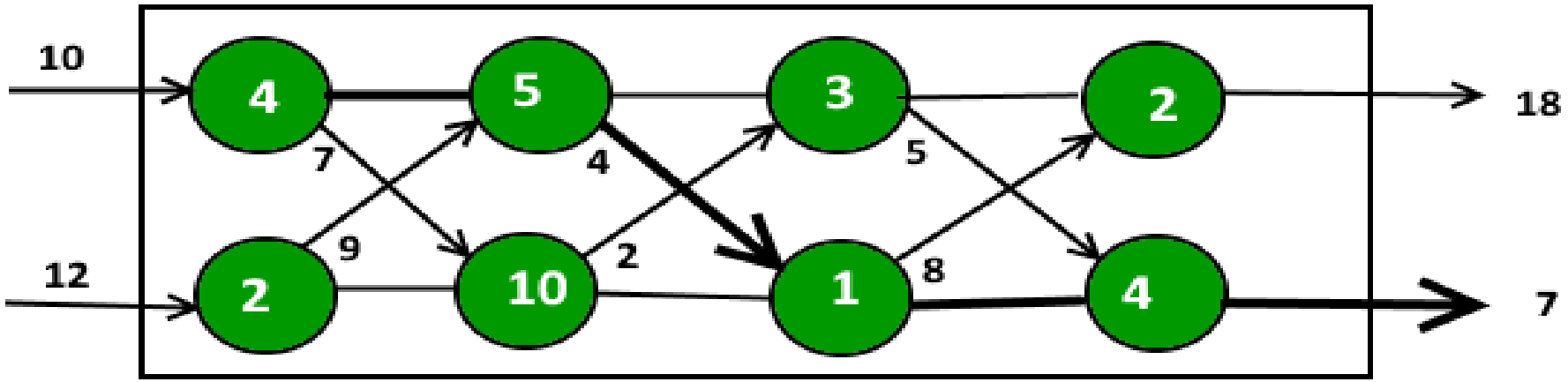
Similarly, the minimum time to reach station S2, j is given by:

$T2(j) = \min((T2(j-1) + a_{2,j}), (T1(j-1) + t_{1,j} + a_{2,j}))$

The total minimum time taken by the car chassis to come out of the factory is given by:

Tmin = min(Time taken to leave station $S_{i,n}$ + Time taken to exit the car factory)

Tmin = min(T1(n) + x_1, T2(n) + x_2)



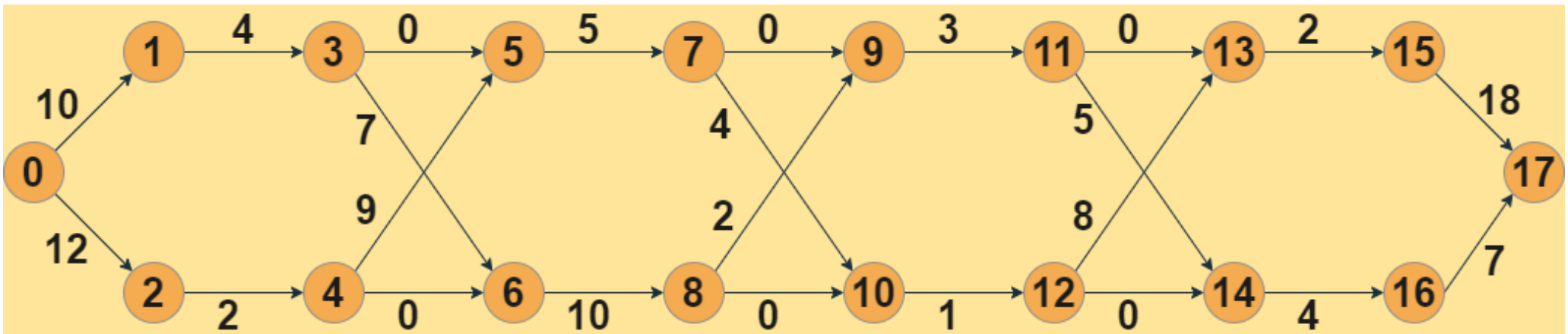
T1(1)	T1(2)	T1(3)	T1(4)	F1
10+4=14	14+5=19 14+9+5=28	19+3=22 24+2+3=29	22+2=24 24+8+2=34	24+18=42
T2(1)	T2(2)	T2(3)	T2(4)	F2
12+2=14	14+10=24 14+7+10=31	24+1=25 19+4+1=24	24+4=28 22+5+4=31	28+7=35

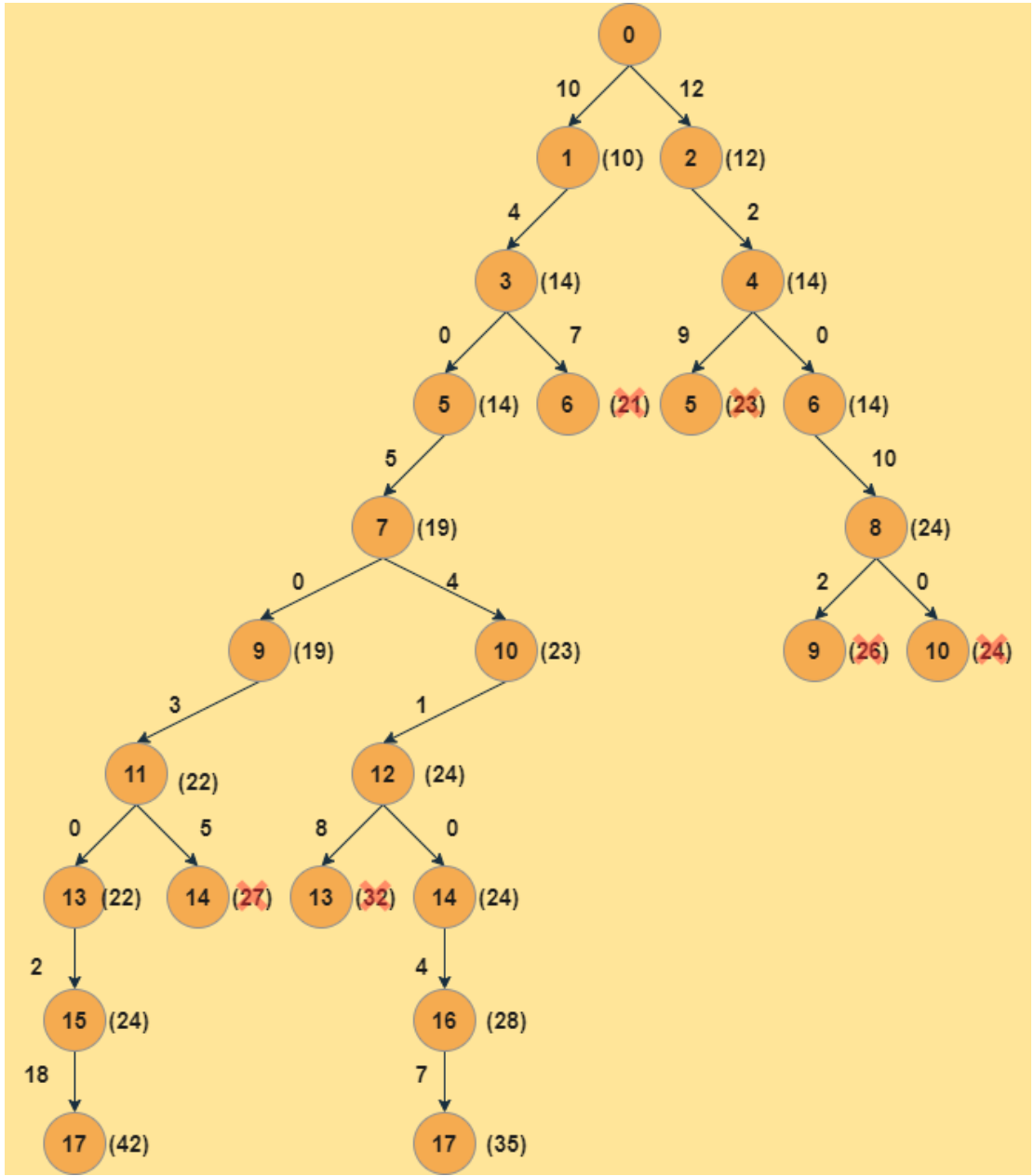
```
int main()
{
    int a[][NUM_STATION] = {{4, 5, 3, 2},
                            {2, 10, 1, 4}};
    int t[][NUM_STATION] = {{0, 7, 4, 5},
                            {0, 9, 2, 8}};
    int e[] = {10, 12}, x[] = {18, 7};
    cout << carAssembly(a, t, e, x);
    return 0;
}
```

Output: 35

```
int carAssembly(int a[][NUM_STATION],
                        int t[][NUM_STATION], int *e, int *x)
{   int T1[NUM_STATION], T2[NUM_STATION], i;
    // time taken to leave first station in line 1
    T1[0] = e[0] + a[0][0];
    // time taken to leave first station in line 2
    T2[0] = e[1] + a[1][0];
    // Fill tables T1[] and T2[] using the above given recursive relations
    for (i = 1; i < NUM_STATION; ++i)
    {  T1[i] = min(T1[i - 1] + a[0][i], T2[i - 1] + t[1][i] + a[0][i]);
       T2[i] = min(T2[i - 1] + a[1][i], T1[i - 1] + t[0][i] + a[1][i]);
    }
    // Consider exit times and return minimum
return min(T1[NUM_STATION - 1] + x[0], T2[NUM_STATION - 1] + x[1]); }
```

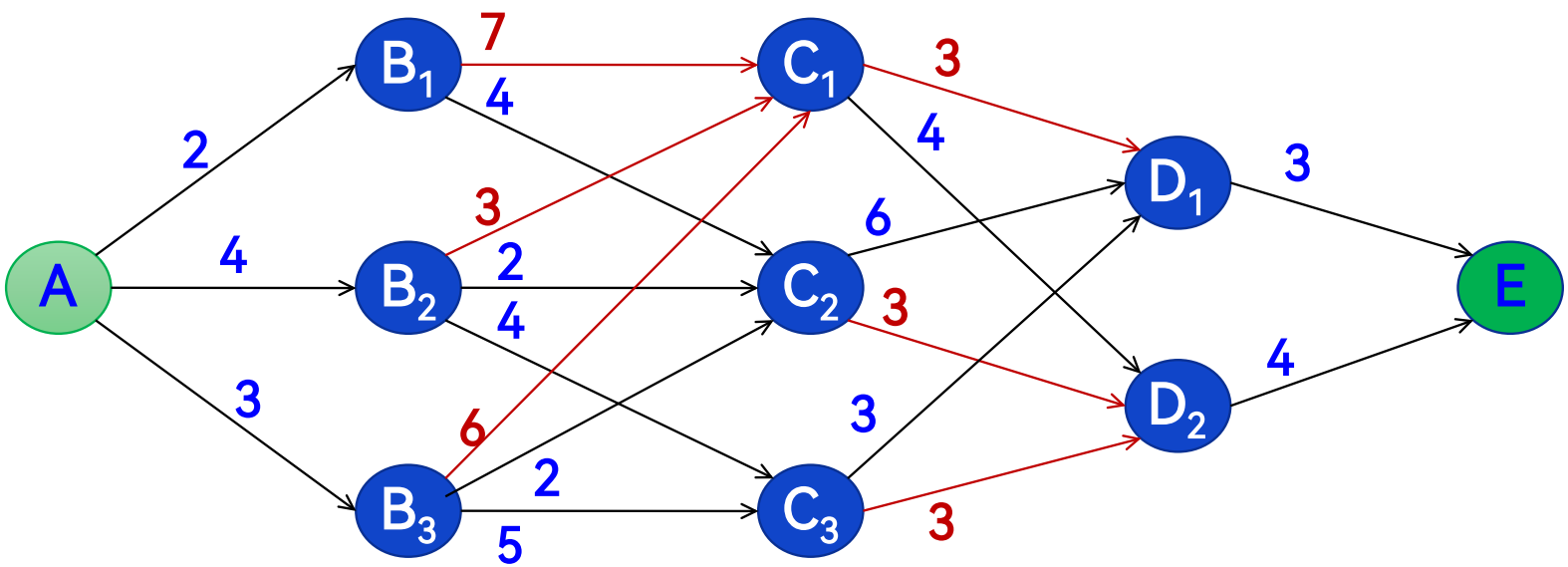
Assembly Line Scheduling



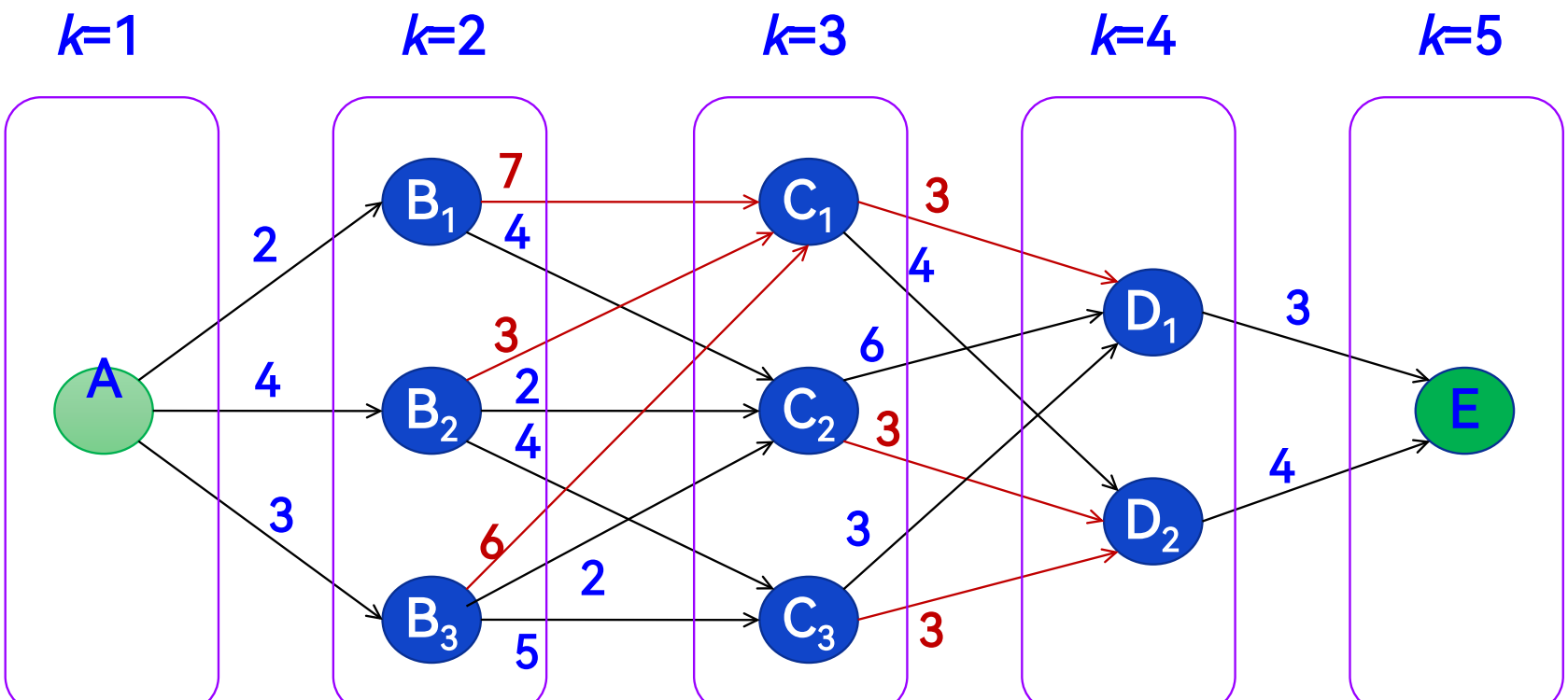

0
10
12
1 (10)
2 (12)
4
2
3 (14)
4 (14)
0
7
9
0
5 (14)
6 (21)
5 (23)
6 (14)
5
10
7 (19)
8 (24)
0
4
2
0
9 (19)
10 (23)
9 (26)
10 (24)
3
1
11 (22)
12 (24)
0
5
8
0
13 (22)
14 (27)
13 (32)
14 (24)
2
4
15 (24)
16 (28)
18
7
17 (42)
17 (35)

动态规划可以用来解决多阶段决策问题的优化方法，把多阶段过程转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解。

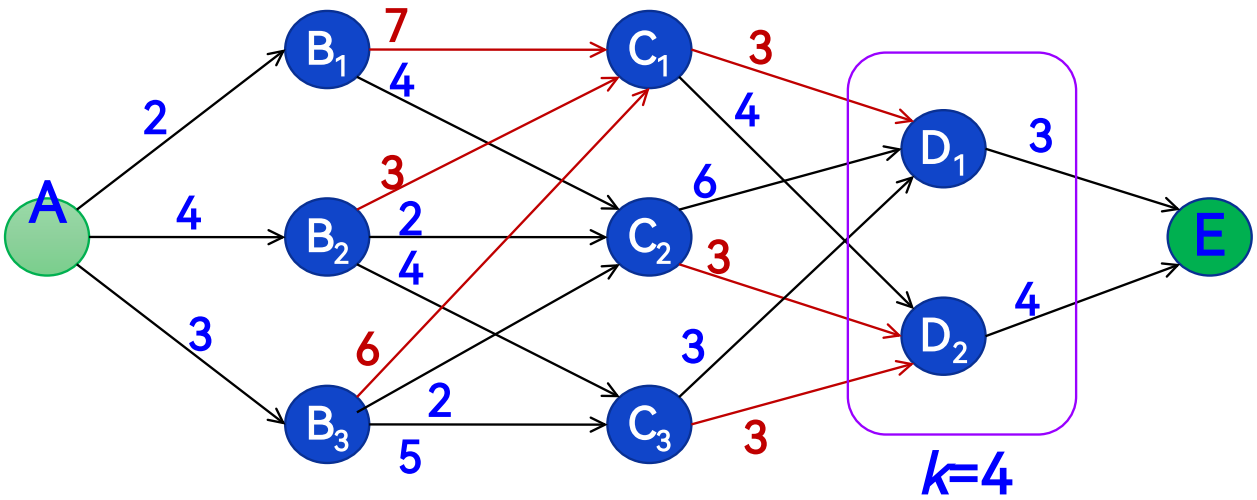
示例： 在*A*处有一水库，现需要从*A*点铺设一条管道到*E*点，边上的数字表示与其相连的两个地点之间所需修建的管道长度，用*c*数组表示，例如$c(A, B_1)=2$。现要找出一条从*A*到*E*的修建线路，使得所需修建的管道长度最短。



在从$A\sim E$的过程中，依据按位置所做的决策的次数及所做决策的先后次序，将问题分为5个阶段，阶段变量用于表示各阶段，这里阶段变量k为1～5，图中第5阶段是虚拟的一个边界阶段。



- 逆序解法
- 顺序解法



求解E→A的过程（用next表示路径上一个顶点的后继顶点）：

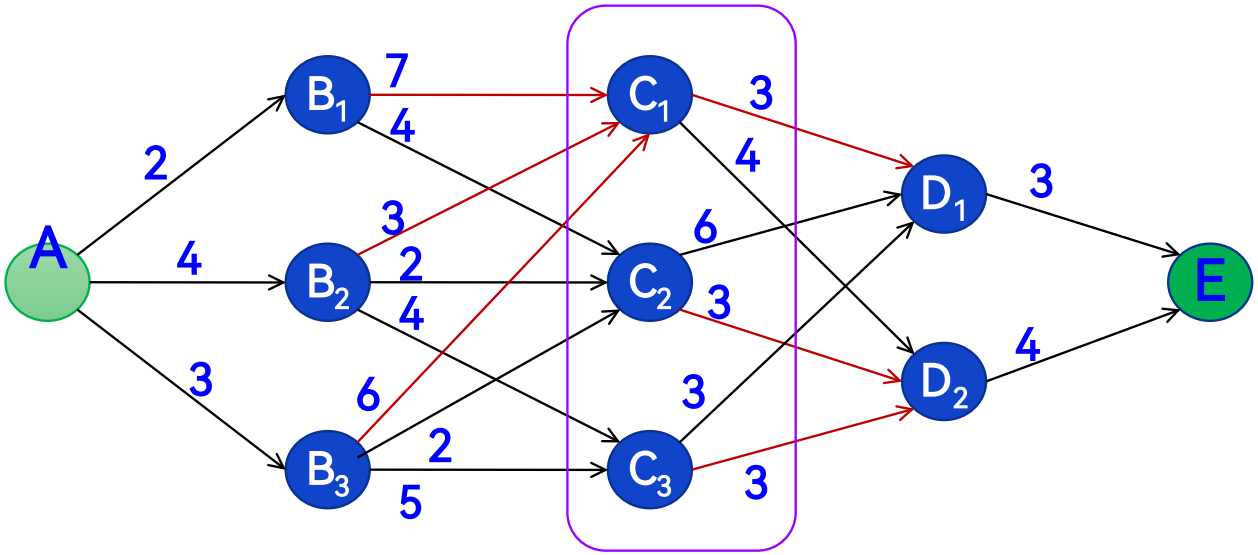
① 第5阶段

$f(E)=0$

② 第4阶段

$f(D_1)=MIN(c(D_1, E)+f(E))=3$， $next(D_1)=E$

$f(D_2)=MIN(c(D_2, E)+f(E))=4$， $next(D_2)=E$



k=3

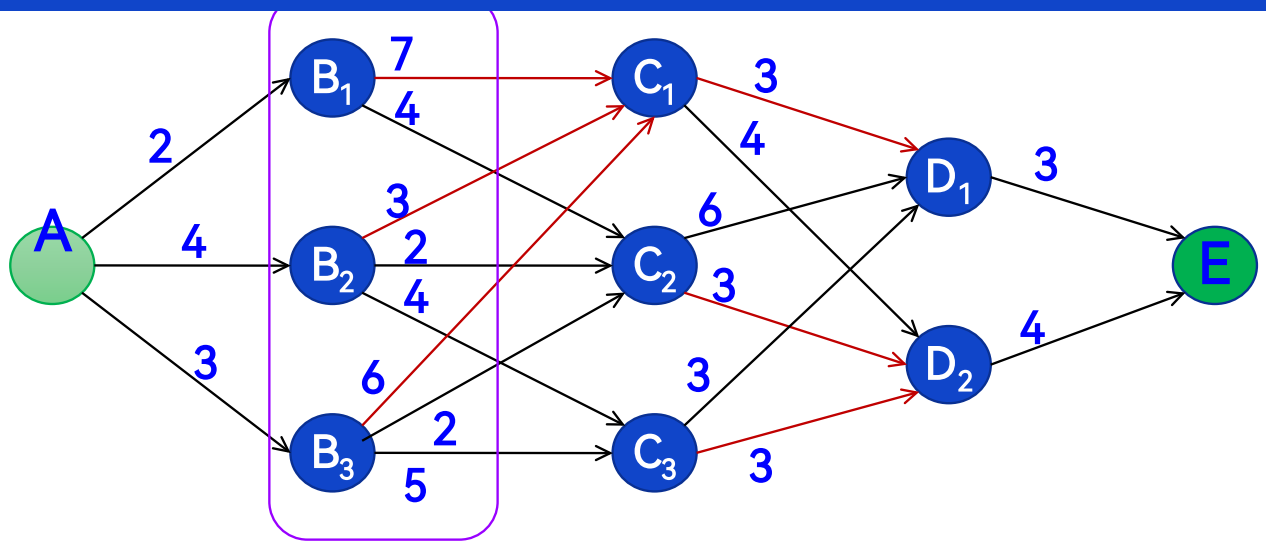
f(D1)=3,
f(D2)=4,

③ 第3阶段

$$f(C_1)=\text{MIN}\begin{pmatrix}c(C_1,D_1)+f(D_1)=6\\ c(C_1,D_2)+f(D_2)=8\end{pmatrix}=6,\quad \text{next}(C_1)=D_1$$

$$f(C_2)=\text{MIN}\begin{pmatrix}c(C_2,D_1)+f(D_1)=9\\ c(C_2,D_2)+f(D_2)=7\end{pmatrix}=7,\quad \text{next}(C_2)=D_2$$

$$f(C_3)=\text{MIN}\begin{pmatrix}c(C_3,D_1)+f(D_1)=6\\ c(C_3,D_2)+f(D_2)=7\end{pmatrix}=6,\quad \text{next}(C_3)=D_1$$



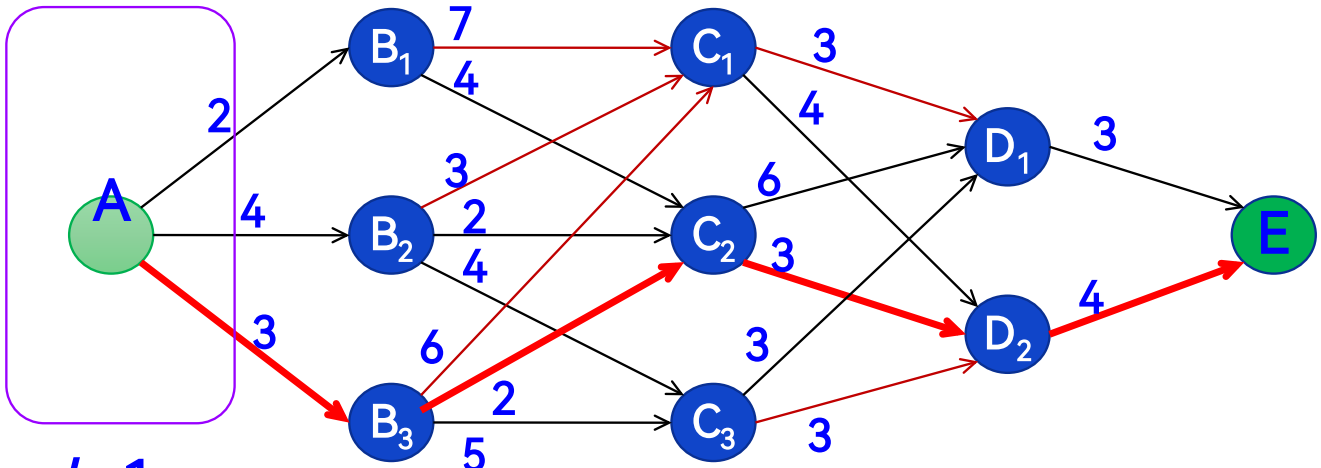
f(C1)=6,
f(C2)=7,
f(C3)=6,

④ 第2阶段　　*k*=2

$$f(B_1)=\mathrm{MIN}\begin{pmatrix} c(B_1,C_1)+f(C_1)=13 \\ c(B_1,C_2)+f(C_2)=11 \\ c(B_1,C_3)+f(C_3)=\infty \end{pmatrix}=11\ ,\ \mathrm{next}(B_1)=C_2$$

$$f(B_2)=\mathrm{MIN}\begin{pmatrix} c(B_2,C_1)+f(C_1)=9 \\ c(B_2,C_2)+f(C_2)=9 \\ c(B_2,C_3)+f(C_3)=10 \end{pmatrix}=9\ ,\ \mathrm{next}(B_2)=C_1$$

$$f(B_3)=\mathrm{MIN}\begin{pmatrix} c(B_3,C_1)+f(C_1)=12 \\ c(B_3,C_2)+f(C_2)=9 \\ c(B_3,C_3)+f(C_3)=11 \end{pmatrix}=9\ ,\ \mathrm{next}(B_3)=C_2$$



k=1

f(B1)=11,
f(B2)=9,
f(B3)=9,

⑤ 第1阶段

$$f(\mathrm{A}) = \mathrm{MIN}\begin{pmatrix} \mathrm{c(A,B_1)} + f(\mathrm{B_1}) = 13 \\ \mathrm{c(A,B_2)} + f(\mathrm{B_2}) = 13 \\ \mathrm{c(A,B_3)} + f(\mathrm{B_3}) = 12 \end{pmatrix} = 12 \quad , \quad \mathrm{next(A)=B_3}$$

由f(A)=12求出的最短路径长度为12

由next(A)=B_3，next(B_3)=C_2，next(C_2)=D_2，next(D_2)=E，推出最短路径为A→B_3→C_2→D_2→E。

```
#include <string.h>
#include <map>
#include <stdio.h>
using namespace std;
#define MAX 21
#define INF 0x3f3f3f3f
//问题表示
int n;                    //顶点个数
int start; //起点编号
int end;  //终点编号
int c[MAX][MAX];                    //存放边长度
int next[MAX];                              //存放最短路径上当前顶点的后继顶点
map<int,char*> vname;               //存放编号对应的顶点名称
int dp[MAX];                        //当前顶点到终点的最短距离
int Count=1;                                //计算步骤
```

```
void Init() //初始化图
{               n=10;
                start=0;
                end=9;
                memset(c,INF,sizeof(c));
                for(int i=0;i<n;i++)
                                dp[i] = -1;
                for(int j=0;j<n;j++)
                                c[j][j] = 0;// 对角线全部置0
                c[0][1]=2; c[0][2]=4; c[0][3]=3;//图8.4的邻接矩阵
                c[1][4]=7; c[1][5]=4;
                c[2][4]=3; c[2][5]=2; c[2][6]=4;
                c[3][4]=5; c[3][5]=2; c[3][6]=5;
                c[4][7]=3; c[4][8]=4;
                c[5][7]=6; c[5][8]=3;
                c[6][7]=3; c[6][8]=3;
                c[7][9]=3;
                c[8][9]=4;
                vname[0]="A";
                vname[1]="B1"; vname[2]="B2"; vname[3]="B3";
                vname[4]="C1"; vname[5]="C2"; vname[6]="C3";
                vname[7]="D1"; vname[8]="D2";
                vname[9]="E";
}
```

```
int f(int s)  //动态规划问题的逆序解法
{   if (dp[s]!=-1) return dp[s];     //若dp[s]已求出，直接返回
        if (s==end)                                                     //找到终点，逆序是从终点向前找
        {       dp[s]=0;
                printf("(%d) f(%s)=0\n",Count++,vname[s]);
                return dp[s];
        }
        else
        {   int cost, mincost=INF,minj;
                for (int j=0;j<n;j++)                                   //查找顶点s的后继顶点
                {   if(c[s][j]!=0&&c[s][j]!=INF)
                        {       cost=c[s][j]+f(j);                      //先求出后继顶点j的f值
                                if(mincost > cost)                      //比较求出从s到终点的最短路径
                                {       mincost=cost;
                                        minj=j;
                                }
                        }
                }
                dp[s]=mincost;
                next[s]=minj;                                                   //当前顶点的后继节点
                printf("(%d) f(%s)=c(%s, %s)+f(%s)=%d, ",Count++,
                                        vname[s],vname[s],vname[minj],vname[minj], dp[s]);
                printf("next(%s)= %s\n",vname[s], vname[minj]);
                return dp[s];
        }
}
```

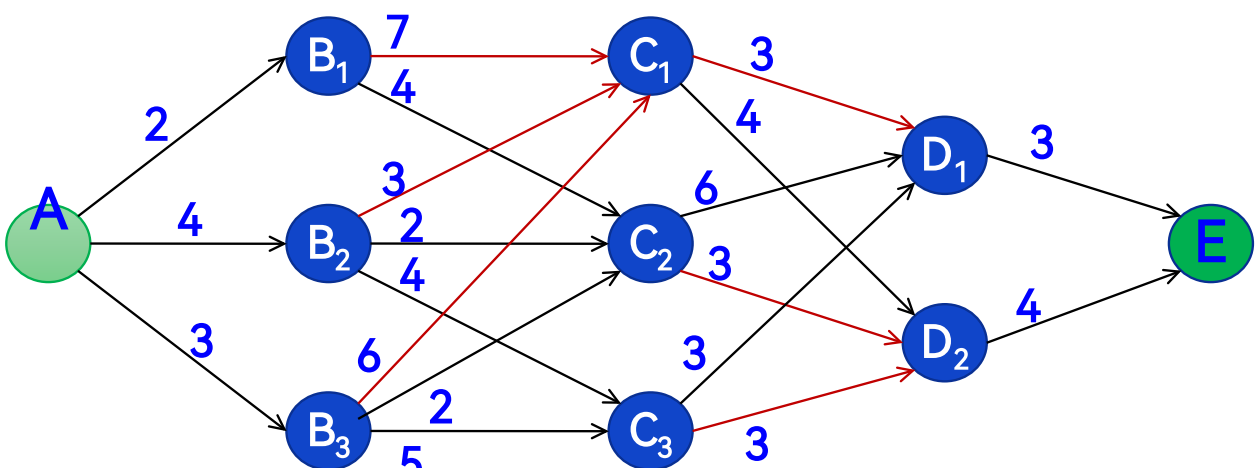
```
int main()
{
	Init();
	printf("%s->%s求解过程\n",vname[end],vname[start]);
	f(start);
	return 0;
}
```

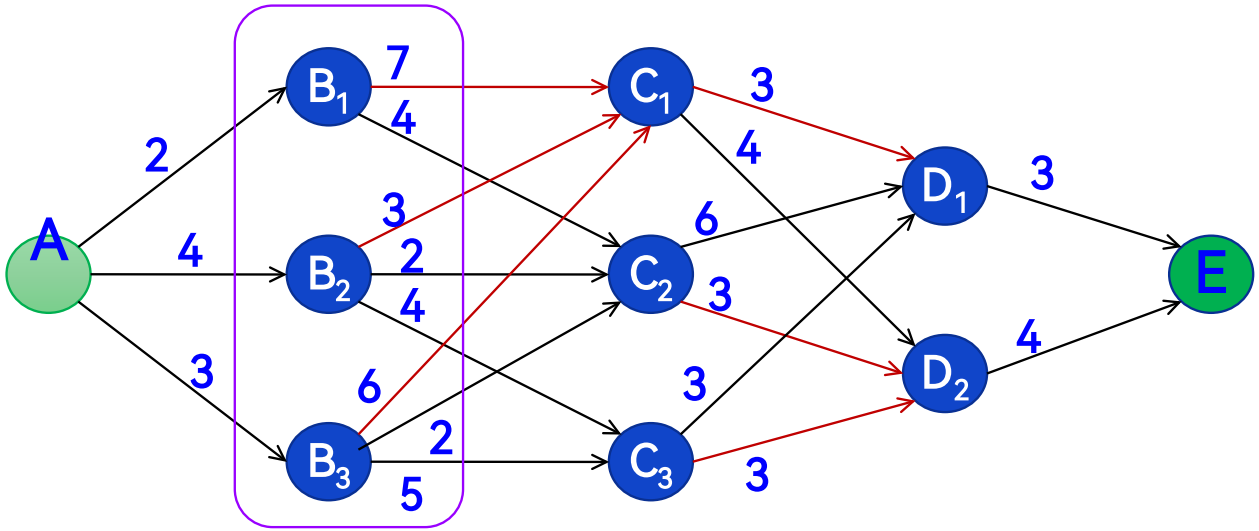
```
E->A求解过程
(1) f(E)=0
(2) f(D1)=c(D1, E)+f(E)=3, next(D1)= E
(3) f(D2)=c(D2, E)+f(E)=4, next(D2)= E
(4) f(C1)=c(C1, D1)+f(D1)=6, next(C1)= D1
(5) f(C2)=c(C2, D2)+f(D2)=7, next(C2)= D2
(6) f(B1)=c(B1, C2)+f(C2)=11, next(B1)= C2
(7) f(C3)=c(C3, D1)+f(D1)=6, next(C3)= D1
(8) f(B2)=c(B2, C1)+f(C1)=9, next(B2)= C1
(9) f(B3)=c(B3, C2)+f(C2)=9, next(B3)= C2
(10) f(A)=c(A, B3)+f(B3)=12, next(A)= B3
```

A→E对应的的状态转移方程如下：

$$f(A) = 0$$

$$f(s) = \min_{\text{存在}<t,s>\text{的有向边}} \{ f(t) + c(t, s) \}$$





用pre表示路径上一个顶点的前驱顶点，其求解$A \to E$的过程：

① 第1阶段

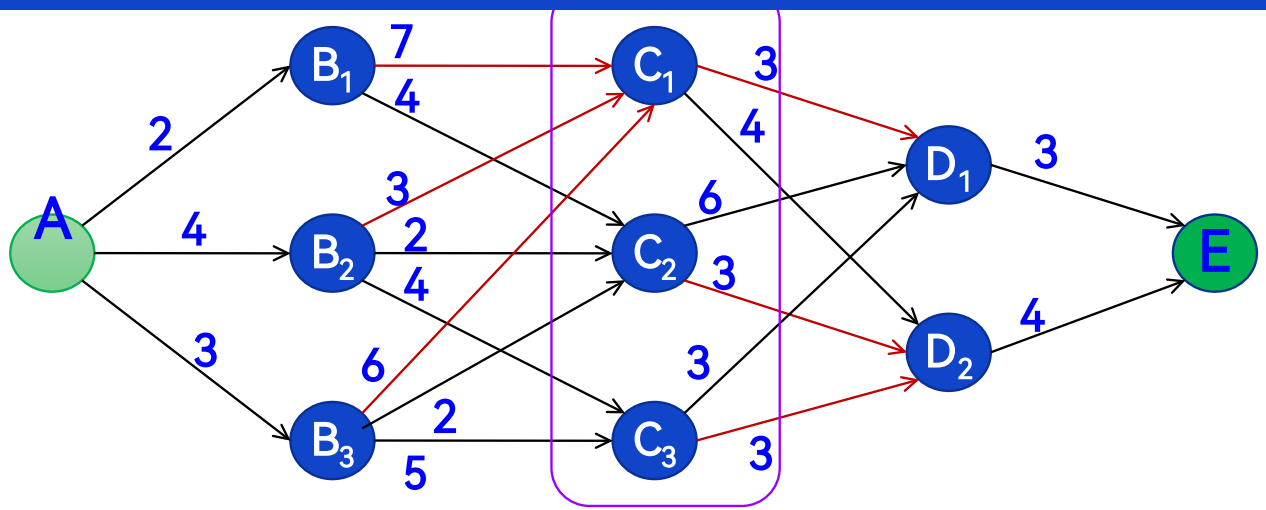
$f(A)=0$

② 第2阶段

$f(B_1)=MIN(f(A)+c(A, B_1))=2$，　$pre(B_1)=A$

$f(B_2)=MIN(f(A)+c(A, B_2))=4$，　$pre(B_2)=A$

$f(B_3)=MIN(f(A)+c(A, B_3))=3$，　$pre(B_3)=A$



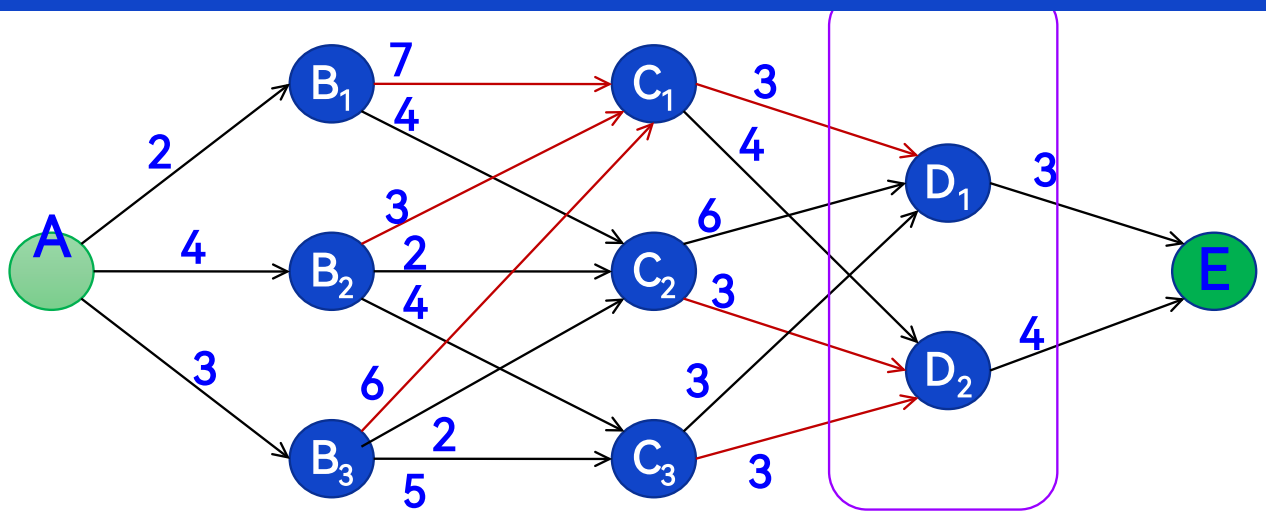
f(B1)=2,
f(B2)=4,
f(B3)=3,

③ 第3阶段

$$f(C_1)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_1)=9 \\ f(B_2)+c(B_2,C_1)=7 \\ f(B_3)+c(B_3,C_1)=9 \end{pmatrix}=7,\quad pre(C_1)=B_2$$

$$f(C_2)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_2)=6 \\ f(B_2)+c(B_2,C_2)=6 \\ f(B_3)+c(B_3,C_2)=5 \end{pmatrix}=5,\quad pre(C_2)=B_3$$

$$f(C_3)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_3)=\infty \\ f(B_2)+c(B_2,C_3)=8 \\ f(B_3)+c(B_3,C_3)=8 \end{pmatrix}=8,\quad pre(C_3)=B_2$$

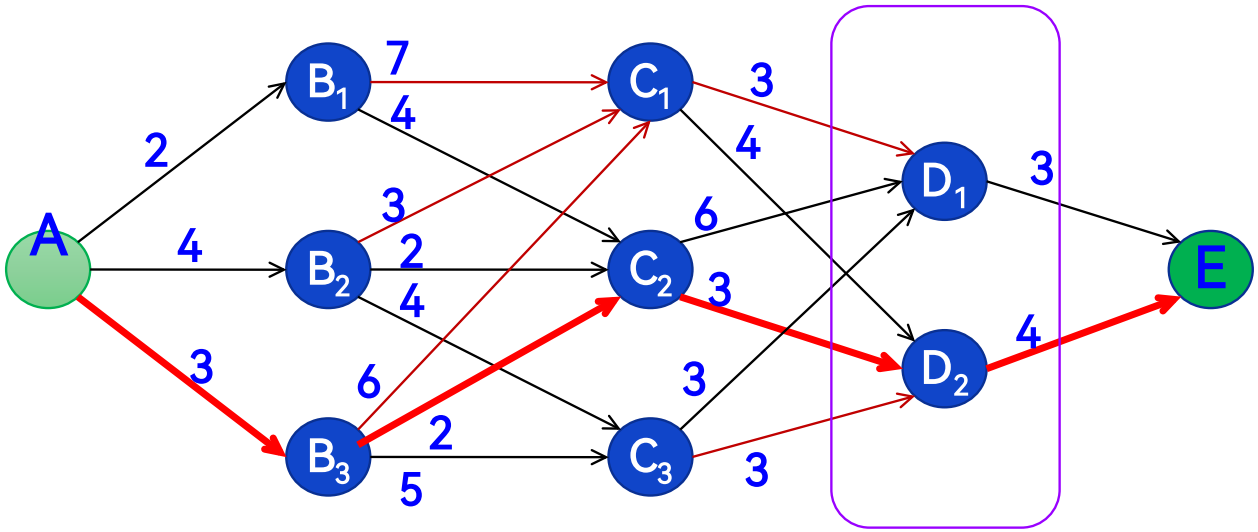


f(C1)=7,
f(C2)=5,
f(C3)=8,

④ 第4阶段

$$f(D_1)=\mathrm{MIN}\begin{pmatrix} f(C_1)+c(C_1,D_1)=10 \\ f(C_2)+c(C_2,D_1)=11 \\ f(C_3)+c(C_3,D_1)=11 \end{pmatrix}=10,\quad \mathrm{pre}(D_1)=C_1$$

$$f(D_2)=\mathrm{MIN}\begin{pmatrix} f(C_1)+c(C_1,D_2)=11 \\ f(C_2)+c(C_2,D_2)=\ \ 8 \\ f(C_3)+c(C_3,D_2)=11 \end{pmatrix}=8,\quad \mathrm{pre}(D_2)=C_2$$



f(D1)=10,
f(D2)=8,

⑤ 第5阶段

$$f(\mathrm{E}) = \mathrm{MIN}\begin{pmatrix} f(\mathrm{D_1}) + \mathrm{c}(\mathrm{D_1},\mathrm{E}) = 13 \\ f(\mathrm{D_2}) + \mathrm{c}(\mathrm{D_2},\mathrm{e}) = 12 \end{pmatrix} = 12,\ \ \mathrm{pre(E)=D_2}$$

由f(E)=12求出的最短路径长度为12

由pre(E)=D_2，pre(D_2)=C_2，pre(C_2)=B_3，pre(B_3)=A，推出最短路径为

A→B_3→C_2→D_2→E。

```
#include <string.h>
#include <stdio.h>
#include <map>
using namespace std;
#define MAX 21
#define INF 0x3f3f3f3f
//问题表示
int n;      //顶点个数
int start; //起点编号
int end;  //终点编号
int c[MAX][MAX];                                    //存放边长度
int pre[MAX];                                       //存放最短路径上当前顶点的前驱顶点
map<int,char *> vname;                              //存编号对应的顶点名称
int dp[MAX];                                          //从起点出发到当前顶点的最短距离

int Count=1;                                        //计算步骤

//这里初始化 Init 函数同逆序解法
```

```
int f(int s)  //动态规划问题的顺序解法
{   if (dp[s]!=-1) return dp[s];//若dp[s]已求出,直接返回
            if(s==start)                                                    //找到起点
            {   dp[s]=0;
                        printf("(%d) f(%s)=0\n",Count++,vname[s]);
                        return dp[s];
            }
            else
            {           int cost, mincost=INF,mini;
                        for(int i=0;i<n;i++)      //查找顶点s的前驱顶点
                        {   if(c[i][s]!=0 && c[i][s]!=INF)
                                    {   cost= f(i)+c[i][s];//先求出前驱顶点i的f值
                                                if(mincost>cost)  //比较求最短路径
                                                {           mincost=cost;
                                                            mini=i;
                                                }
                                    }
                        }
                        dp[s]=mincost;
                        pre[s]=mini;                                        //设置当前顶点 s 的前驱顶点为 mini
                        printf("(%d) f(%s)=f(%s)+c(%s,%s)=%d,",Count++,
                                    vname[s],vname[mini],vname[mini],vname[s],dp[s]);
                        printf("pre(%s)=%s\n",vname[s],vname[mini]);
                        return dp[s];
            }
}
```

```
int main()
{
	Init();
	printf("%s->%s 求解过程\n",vname[start],vname[end]);
	f(end);
	return 0;
}
```

```
A->E 求解过程
(1) f(A)=0
(2) f(B1)=f(A)+c(A,B1)=2,pre(B1)=A
(3) f(B2)=f(A)+c(A,B2)=4,pre(B2)=A
(4) f(B3)=f(A)+c(A,B3)=3,pre(B3)=A
(5) f(C1)=f(B2)+c(B2,C1)=7,pre(C1)=B2
(6) f(C2)=f(B3)+c(B3,C2)=5,pre(C2)=B3
(7) f(C3)=f(B2)+c(B2,C3)=8,pre(C3)=B2
(8) f(D1)=f(C1)+c(C1,D1)=10,pre(D1)=C1
(9) f(D2)=f(C2)+c(C2,D2)=8,pre(D2)=C2
(10) f(E)=f(D2)+c(D2,E)=12,pre(E)=D2
```

【问题描述】给定一个有n（n≥1）个整数的序列，要求求出其中最大连续子序列的和。

例如：

序列（-2，11，-4，13，-5，-2）的最大子序列和为20

序列（-6，2，4，-7，5，3，2，-1，6，-9，10，-2）的最大子序列和为16。

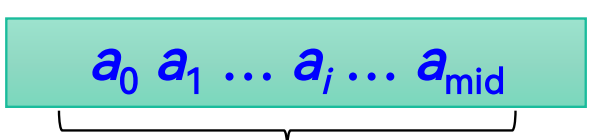
规定一个序列最大连续子序列和至少是0，如果小于0，其结果为0。

$$\bar{v} \int_{0}^{+\infty} v \cdot f(v) \cdot dv$$

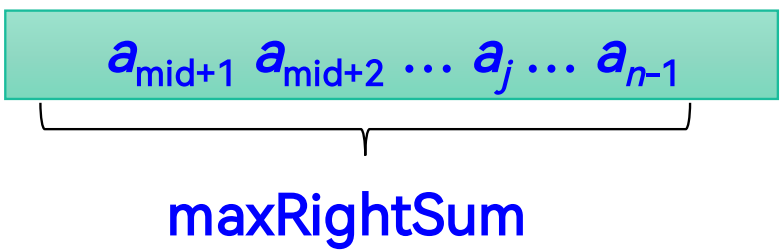
【问题求解】对于含有n个整数的序列$a[0..n-1]$，若$n=1$，表示该序列仅含一个元素，如果该元素大于0，则返回该元素；否则返回0。

若$n>1$，采用分治法求解最大连续子序列时，取其中间位置$mid=\lfloor (n-1)/2 \rfloor$，该子序列只可能出现3个地方。

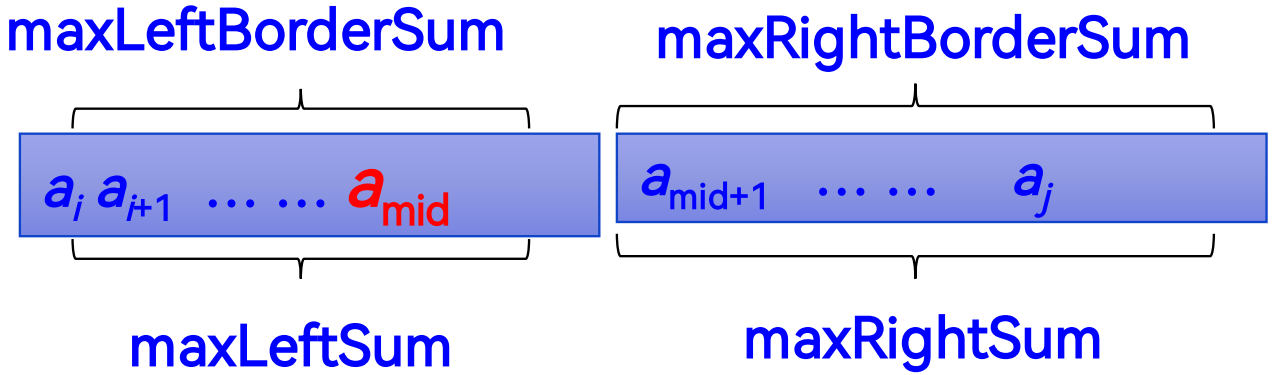
（1）该子序列完全落在左半部即$a[0..mid]$中。采用递归求出其最大连续子序列和maxLeftSum。



（2）该子序列完全落在右半部即a[mid+1..n-1]中。采用递归求出其最大连续子序列和maxRightSum。



（3）该子序列跨越序列*a*的中部而占据左右两部分。



结果：max3(maxLeftSum,
maxRightSum,
maxLeftBorderSum+maxRightBorderSum)

```
long maxSubSum(int a[], int left, int right)
//求a[left..high]序列中最大连续子序列和
{  int i, j;
   long maxLeftSum, maxRightSum;
   long maxLeftBorderSum, leftBorderSum;
   long maxRightBorderSum, rightBorderSum;

   if (left==right)                  //子序列只有一个元素时
   {      if (a[left]>0)             //该元素大于0时返回它
                  return a[left];
            else                                  //该元素小于或等于0时返回0
                        return 0;
   }
```

```
    int mid=(left+right)/2;                    //求中间位置
    maxLeftSum=maxSubSum(a，left，mid);          //求左边
    maxRightSum=maxSubSum(a，mid+1，right);      //求右边
    maxLeftBorderSum=0，leftBorderSum=0;
    for (i=mid;i>=left;i--)                                  //求出以左边加上a[mid]元素
    {  leftBorderSum+=a[i];                                  //构成的序列的最大和
     if (leftBorderSum>maxLeftBorderSum)
            maxLeftBorderSum=leftBorderSum;
    }
    maxRightBorderSum=0，rightBorderSum=0;
    for (j=mid+1;j<=right;j++)                 //求出a[mid]右边元素
    {  rightBorderSum+=a[j];                   //构成的序列的最大和
       if (rightBorderSum>maxRightBorderSum)
            maxRightBorderSum=rightBorderSum;
    }
    return max3(maxLeftSum，maxRightSum，
                    maxLeftBorderSum+maxRightBorderSum);
}
```

【算法分析】设求解序列$a[0..n-1]$最大连续子序列和的执行时间为$T(n)$，第（1）、（2）两种情况的执行时间为$T(n/2)$，第（3）种情况的执行时间为$O(n)$，所以得到以下递推式：

$$T(n)=1 \quad \text{当} n=1$$
$$T(n)=2T(n/2)+n \quad \text{当} n>1$$

容易推出，$T(n)=O(n\log_2 n)$。

蛮力解法1：设含有n个整数的序列$a[0..n-1]$，其中任何连续子序列$a[i..j]$（$i \le j$，$0 \le i \le n-1$，$i \le j \le n-1$）求出它的所有元素之和thisSum。

通过比较将最大值存放在maxSum中，最后返回maxSum。

$a[0]\ a[1]\ \dots\ a[i]\ a[i+1]\ \dots\ a[j]\ a[j+1]\ \dots\ a[n-1]$

thisSum

MAX

maxSum

i：$0 \sim n-1$

j：$i \sim n-1$

k：$i \sim j$

5.8 最大连续子序列和问题

```
int maxSubSum1(int a[],int n)
{  int i,j,k;
   int maxSum=a[0],thisSum;
   for (i=0;i<n;i++)                    //两重循环穷举所有的连续子序列
   {  for (j=i;j<n;j++)
      {  thisSum=0;
         for (k=i;k<=j;k++)
                  thisSum+=a[k];
              if (thisSum>maxSum)  //通过比较求最大连续子序列之和
                  maxSum=thisSum;
       }
    }
    return maxSum;
}
```

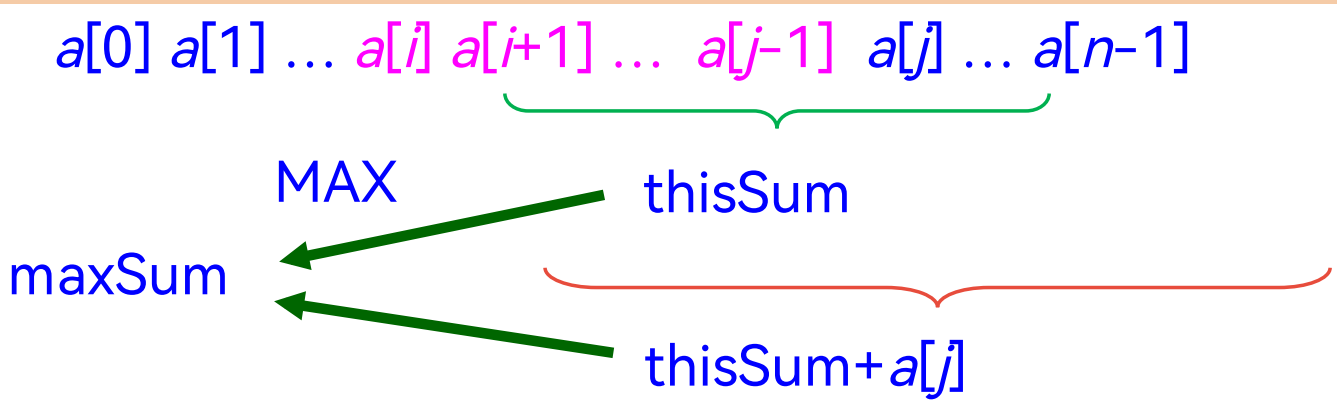
maxSubSum1(*a*,*n*)算法中用了三重循环，所以有：

$$T(n)=\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}\sum_{k=i}^{j}1=\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}(j-i+1)=\frac{1}{2}\sum_{i=0}^{n-1}(n-i)(n-i+1)$$

$=O(n^3)$。

解法2：改进前面的解法，在求两个相邻子序列和时，它们之间是关联的。

例如$a[0..3]$子序列和$=a[0]+a[1]+a[2]+a[3]$，$a[0..4]$子序列和$=a[0]+a[1]+a[2]+a[3]+a[4]$，在前者计算出来后，求后者时只需在前者基础上加以$a[4]$即可，没有必须每次都重复计算。从而提高了算法效率。



i：0 ~ $n-1$

j：i ~ $n-1$

```
int maxSubSum2(int a[],int n)
{  int i,j;
   int maxSum=a[0],thisSum;
   for (i=0;i<n;i++)
   {  thisSum=0;
      for (j=i;j<n;j++)
      {  thisSum+=a[j];  //maxSum已经包含了a[i..j-1]的最大和
                 if (thisSum>maxSum)
                     maxSum=thisSum;
         }
   }
   return maxSum;
}
```

maxSubSum2(*a*,*n*)算法中只有两重循环，容易求出T(*n*)=O(n^2)。

解法3：更一步改进解法2。

如果扫描中遇到负数，当前子序列和thisSum将会减小，若thisSum为负数，表明前面已经扫描的那个子序列可以抛弃了，则放弃这个子序列，重新开始下一个子序列的分析，并置thisSum为0。

若这个子序列和thisSum不断增加，那么最大子序列和maxSum也不断增加。

```
int maxSubSum3(int a[],int n)
{  int i,maxSum=0,thisSum=0;
   for (i=0;i<n;i++)
   {  thisSum+=a[i];
      if (thisSum<0)        //若当前子序列和为负数，重新开始下一子序列
                thisSum=0;
      if (maxSum<thisSum)   //比较求最大连续子序列和
                maxSum=thisSum;
   }
   return maxSum;
}
```

显然该算法中仅扫描*a*一次，其算法的时间复杂度为O(*n*)。

【问题求解】对于含有n个整数的序列a，设

$b_j=\mathrm{MAX}\{a_i+a_{i+1}+\ldots+a_j\}$ （$1\le j\le n$）

表示$a[1..j]$的前j个元素中的最大连续子序列和，则b_{j-1}表示$a[1..j-1]$的前$j-1$个元素中的最大连续子序列和。

当$b_{j-1}>0$时，$b_j=b_{j-1}+a_j$，当$b_{j-1}\leq 0$时，放弃前面选取的元素，从a_j开始重新选起，$b_j=a_j$。用一维动态规划数组dp存放b，对应的状态转移方程如下：

dp[0]=0	边界条件
dp[j]=MAX{dp[j-1] +a_j，a_j}	$1\leq j\leq n$

则序列a的最大连续子序列和等于dp[j]（$1\leq j\leq n$）中的**最大者**。

```
//问题表示
int n=6;
int a[]={0，-2，11，-4，13，-5，-2};          //a数组不用下标为0的元素
//求解结果表示
int dp[MAXN];
void maxSubSum()                              //求dp数组
{  dp[0]=0;
   for (int j=1;j<=n;j++)
            dp[j]=max(dp[j-1]+a[j]，a[j]);
}
```

dp[0] = 0

for(int j=1; j≤n; j++)

dp[i] = max(dp[i-1]+a[i], a[i])

【算法分析】maxSubSum() 的时间复杂度为O(*n*) 。

5.9 求解最长递增子序列问题

【问题描述】给定一个无序的整数序列$a[0..n-1]$，求其中最长递增子序列的长度。

例如，$a[]=\{2，1，5，3，6，4，8，9，7\}$，$n=9$，其最长递增子序列为{1，3，4，8，9}，结果为5。

最长

【问题求解】设计动态规划数组为一维数组dp，dp[*i*]表示*a*[0..*i*]中以*a*[*i*]结尾的最长递增子序列的长度。对应的状态转移方程如下：

dp[*i*]=1　0≤i≤n-1

dp[*i*]=max(dp[*i*]，dp[*j*]+1) 若a[i]>a[j]，0≤i≤n-1，0≤j≤i-1

求出dp后，其中最大元素即为所求。

```
//问题表示
int a[]={2，1，5，3，6，4，8，9，7};
int n=sizeof(a)/sizeof(a[0]);
//求解结果表示
int ans=0;
int dp[MAX];

void solve(int a[]，int n)
{  int i，j;
   for(i=0;i<n;i++)
   {  dp[i]=1;
      for(j=0;j<i;j++)
      {  if (a[i]>a[j])
            dp[i]=max(dp[i]，dp[j]+1);
      }
   }
   ans=dp[0];
   for(i=1;i<n;i++)
     ans=max(ans，dp[i]);
}
```

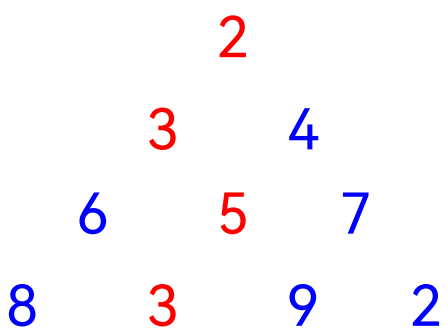
```
for(int i=0;i<n;i++)
{  dp[i]=1;
   for(j=0;j<i;j++)
   {  if (a[i]>a[j])
         dp[i]=max{dp[i]，dp[j]+1};
```

【算法分析】solve()算法中含两重循环，时间复杂度为$O(n^2)$。

5.10 求解三角形最小路径

【问题描述】给定高度为n的一个整数三角形，找出从顶部到底部的最小路径和，只能向先移动相邻的结点。首先输入n，接下来的1～n行，第i行输入i个整数，输出分为2行，第一行为最小路径，第2行为最小路径和。

例如，下图是一个n=4的三角形，输出的路径是2 3 5 3，最小路径是13。



【问题求解】将三角形采用二维数组a存放，前面的三角形对应的二维数组如下：

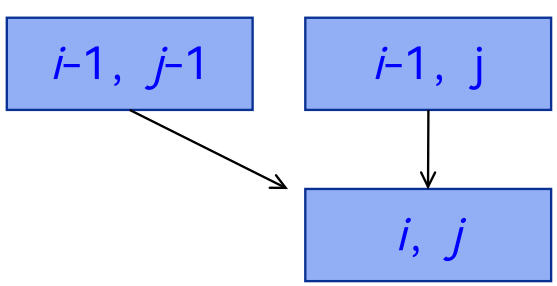
2

3 4

6 5 7

8 3 9 2

从顶部到底部查找最小路径，那么结点（i，j）的前驱结点只有（i-1，j-1）和（i-1，j）两个：



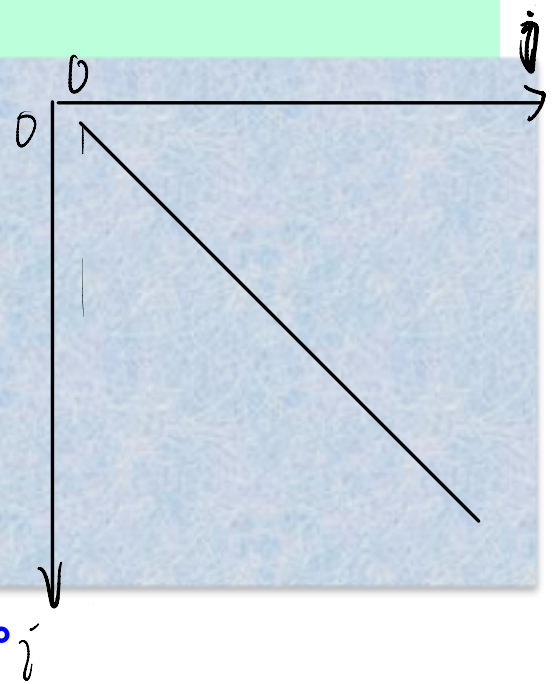
dp[i][j]表示从顶部a[0][0]查找到（i，j）结点时的最小路径和。显然这里有两个边界，即第1列和对角线，达到它们中结点的路径只有一条而不是常规的两条。所以状态转移方程如下：

dp[0][0]=a[0][0] 初始化　　顶部边界

dp[i][0]=dp[i-1][0]+a[i][0]　　考虑第1列的边界，1≤i≤n-1

dp[i][i]=dp[i-1][i-1]+a[i][i]　　考虑对角线的边界，1≤i≤n-1

dp[i][j]=min(dp[i-1][j-1]，dp[i-1][j])+a[i][j]　i>1的其他有两条达到路径的结点



最后求出的最小路径和ans=min(dp[n-1][j])，以及对应的列号k。

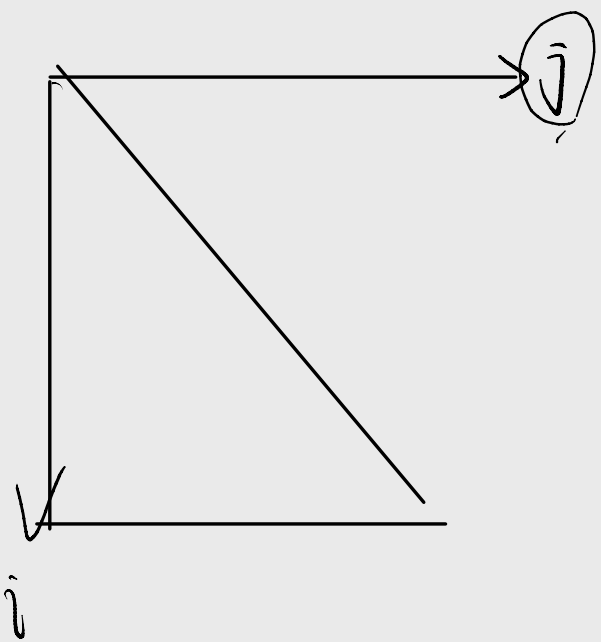
由dp求一条路径?

用pre[i][j]表示查找到（i，j）结点时最小路径上的前驱结点，由于前驱结点只有两个，即（i-1，j-1）和（i-1，j），用pre[i][j]记录前驱结点的列号即可。

在求出ans后，通过pre[n-1][k]反推求出反向路径，最后正向输出该路径。

```
//问题表示
int a[MAXN][MAXN];
int n;
//求解结果表示
int ans=0;
int dp[MAXN][MAXN];
int pre[MAXN][MAXN];
```

```
int Search()                                    //求最小和路径ans
{  int i，  j;
   dp[0][0]=a[0][0];
   for(i=1;i<n;i++)                  //考虑第1列的边界
   {  dp[i][0]=dp[i-1][0]+a[i][0];
      pre[i][0]=i-1;
   }
   for (i=1;i<n;i++)                 //考虑对角线的边界
   {  dp[i][i]=a[i][i]+dp[i-1][i-1];
      pre[i][i]=i-1;
   }
```



```
for(i=2;i<n;i++)                    //考虑其他有两条达到路径的结点
{  for(j=1;j<i;j++)
   {  if(dp[i-1][j-1]<dp[i-1][j])
      {  pre[i][j]=j-1;
         dp[i][j]=a[i][j]+dp[i-1][j-1];
      }
      else
      {  pre[i][j]=j;
         dp[i][j]=a[i][j]+dp[i-1][j];
      }
   }
}
```

```
    ans=dp[n-1][0];
    int k=0;
    for (j=1;j<n;j++)                //求出最小ans和对应的列号k
    {  if (ans>dp[n-1][j])
       {  ans=dp[n-1][j];
          k=j;
       }
    }
    return k;
}
```

```
int main()
{  int k;
   memset(pre，0，sizeof(pre));
   memset(dp，0，sizeof(dp));
   scanf("%d"，&n);                                    //输入三角形的高度
   for (int i=0;i<n;i++)                          //输入三角形
      for (int j=0;j<=i;j++)
         scanf("%d"，&a[i][j]);
   k=Search();                                          //求最小路径和
   Disppath(k);                                   //输出正向路径
   printf("%d\n"，ans);                          //输出最小路径和
   return 0;
}
```

【算法分析】Search()算法中有i从2到$n-1$、j从1到$i-1$的两重循环，容易求出时间复杂度为$O(n^2)$。